

对于顺序存储的线性表，其算法的时间复杂度为 $O(1)$ 的运算应该是（ ）。

- A. 将 n 个元素从小到大排序
- B. 删除第 i 个元素（ $1 \leq i \leq n$ ）
- C. 改变第 i 个元素的值（ $1 \leq i \leq n$ ）
- D. 在第 i 个元素（ $1 \leq i \leq n$ ）后插入一个元素

答案：C

链表不具有的特点是（ ）。

- A. 可随机访问任一元素**
- B. 插入删除不需要移动元素**
- C. 不必事先估计存储空间**
- D. 所需空间与线性表长度成正比**

答案： A

线性表 L 在 () 情况下适用于使用链式结构实现。

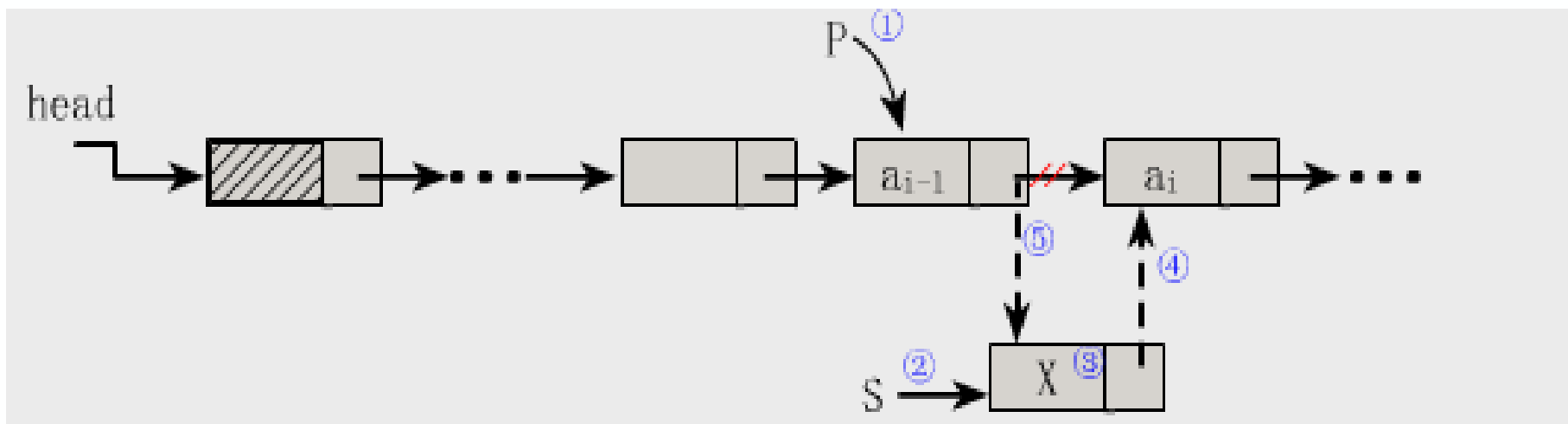
- A. 需经常修改 L 中的结点值**
- B. 需不断对 L 进行插入删除**
- C. L 中含有大量的结点**
- D. L 中结点结构复杂**

答案： B

以下说法**错误**的是（ ）。

- A. 求表长、定位这两种运算在采用顺序存储结构时实现的效率不比采用链式存储结构时实现的效率低
- B. 顺序存储的线性表可以随机存取
- C. 由于顺序存储要求连续的存储区域，所以在存储管理上不够灵活
- D. 线性表的链式存储结构优于顺序存储结构

答案：D 链式存储结构和顺序存储结构各有优缺点，有不同的适用场合。



单链表的插入核心代码：① $S \rightarrow \text{next} = P \rightarrow \text{next};$

② $P \rightarrow \text{next} = S;$

删除数据域为 a_i 的结点的核心代码：

$P \rightarrow \text{next} = P \rightarrow \text{next} \rightarrow \text{next};$

(A, B, C, D, , Z)

数据结构

(非数值问题)

数据的逻辑结构

线性结构

线性表

栈、队列 P62

串、数组

非线性结构

树形结构

图形结构

数据的存储结构

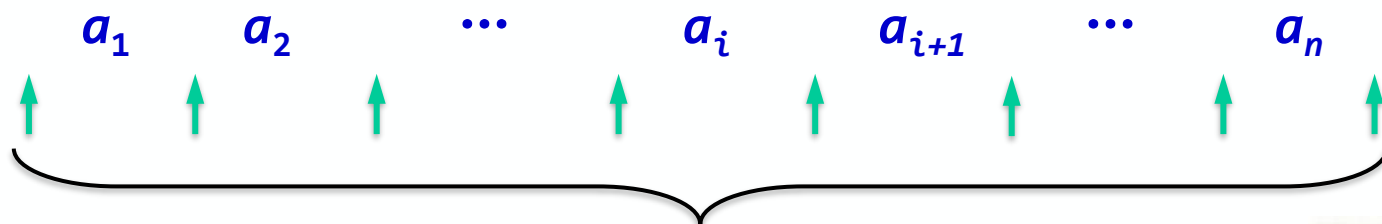
顺序存储 (数组)

链式存储 (指针)

数据的操作：插入、删除、修改、查找、排序

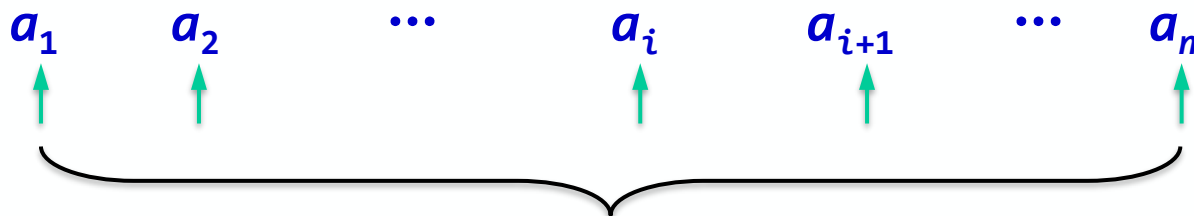
上节内容回顾

■ 顺序表的插入：



在线性表L中共有 $n+1$ 个可以插入元素的地方

■ 顺序表的删除：



在线性表L中共有 n 个可以删除元素的地方



随机存取

第3章 栈和队列

3.1 栈的定义、特点

3.2 案例引入

3.3 栈的表示和操作的实现

3.4 栈与递归

3.5 队列的定义、特点、表示和操作

3.6 案例分析与实现

本章教学内容及重点难点

教学内容：

- (1) 栈和队列的定义及特点；
- (2) 存储结构的表示与实现；
- (3) 栈与递归；
- (4) 循环队列；

教学重点：

- (1) 两种存储结构；
- (2) 基本操作；

教学难点：

- (1) 判空和判满的条件；
 - (2) 栈与递归；
 - (3) 循环队列。
-

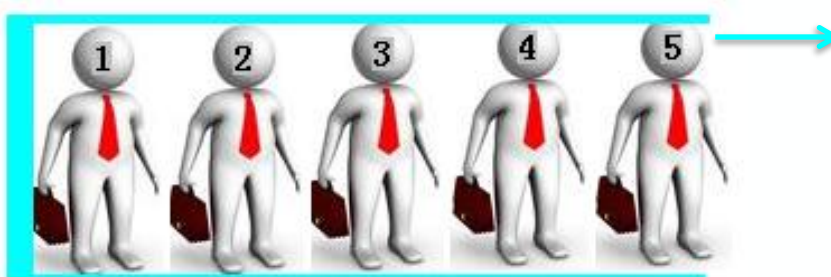
3.1 栈的定义、特点 (P62)



假设死胡同的宽度
恰好只够站一个人



走进死胡同的5人要
按相反次序退出



死胡同就是一个栈！

栈的主要特点是“**先进后出**”，即后进栈的元素先出栈。栈也称为**先进后出表**。

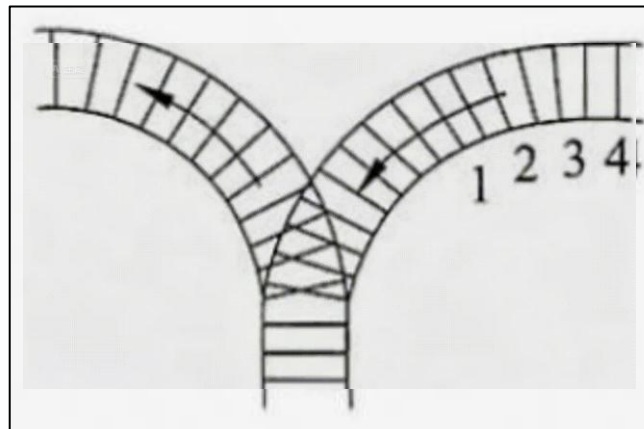
栈——先进后出



洗脏盘子

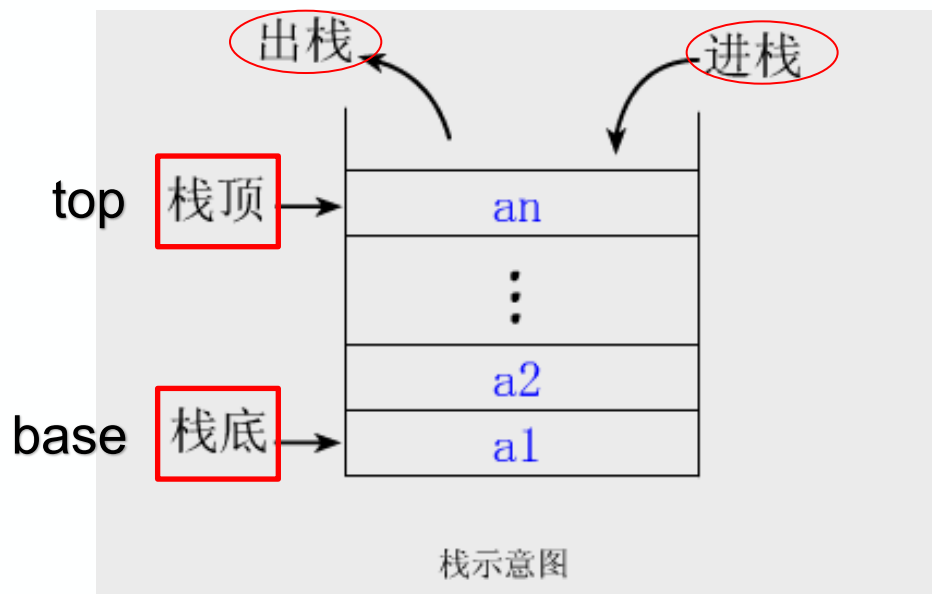


子弹压入弹匣



Y型铁路调度线

3.1 栈的定义、特点 (P62)

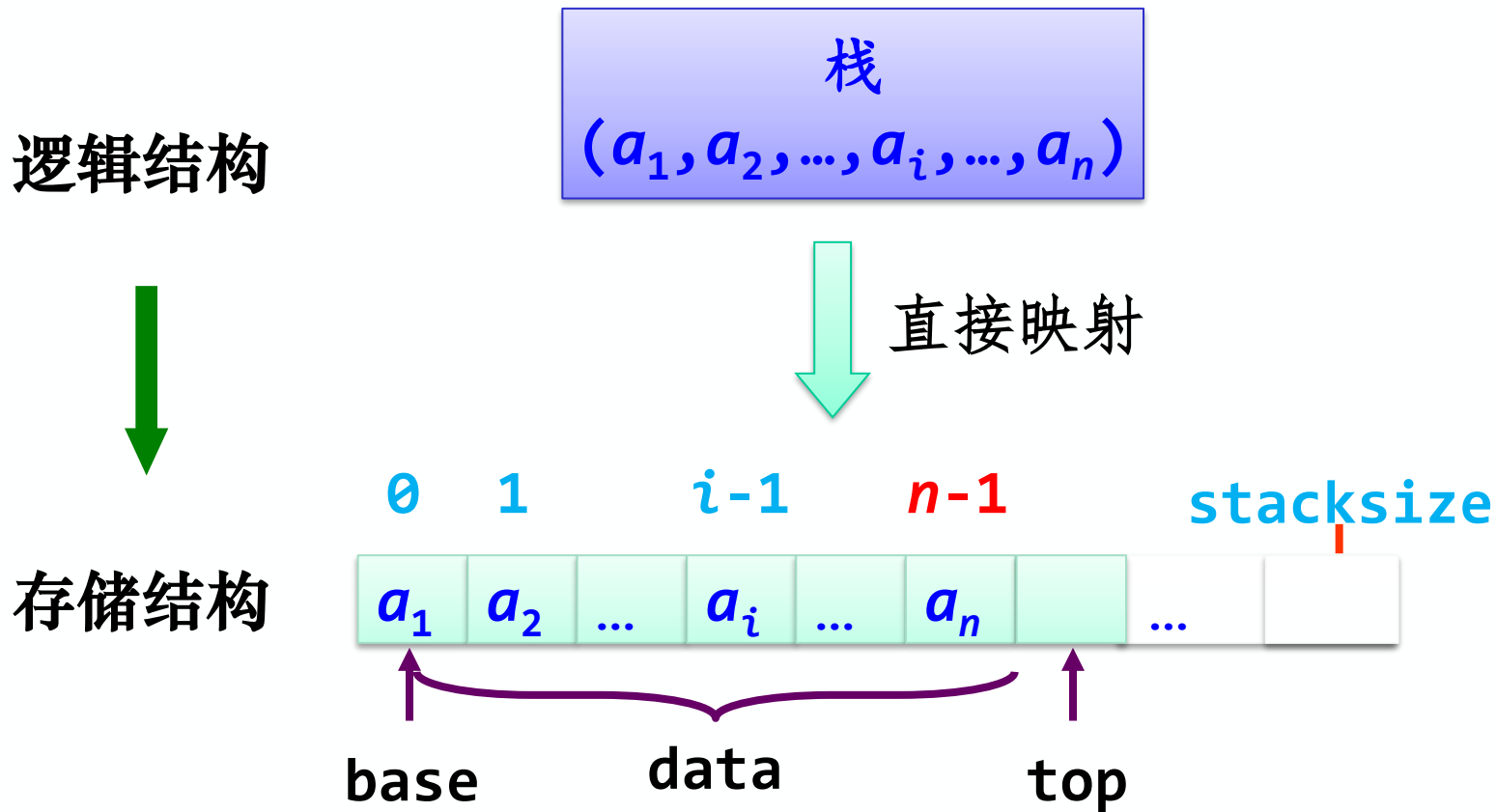


- (1) 定义：只能在**栈顶**进行插入和删除运算的**线性结构**。
- (2) 存储结构：用**顺序栈**或**链栈**存储均可，但以顺序栈更常见。
- (3) 运算规则：只能在**栈顶**运算，**先进后出(FILO)**或**后进先出(LIFO)**的原则。
- (4) 基本操作：有**建栈**、**入栈**、**判断栈满**、**出栈**、**判断栈空**、**读栈顶元素值**等。
push pop

3.3.2 栈的顺序存储结构(P66)

利用一组地址连续的存储单元依次存放自栈底到栈顶的数据元素。

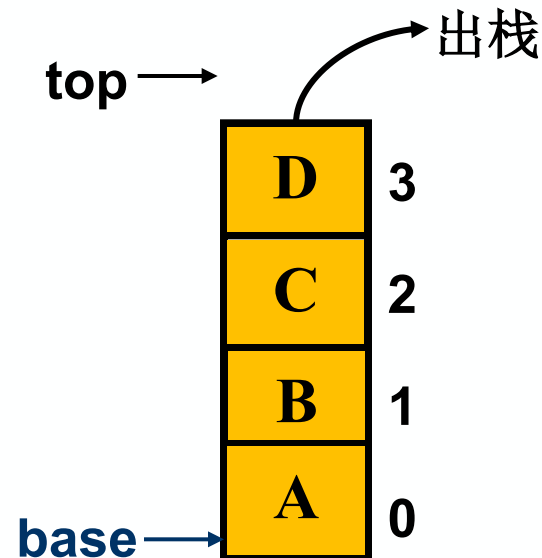
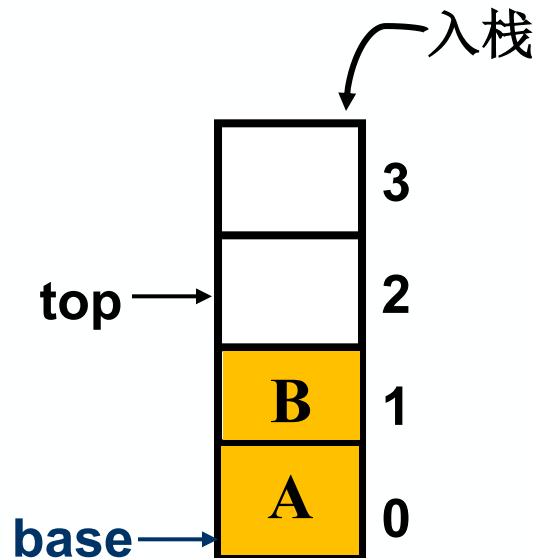
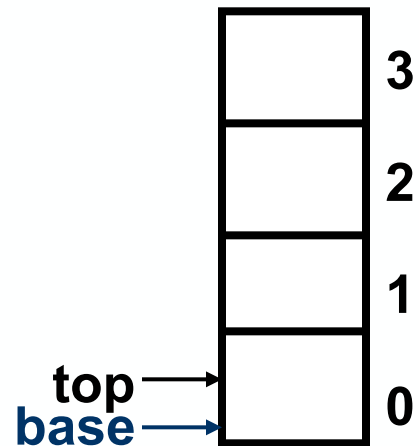
为了方便，通常 top 指示的是栈顶元素之后的下标地址



顺序栈的示意图

顺序栈入栈和出栈(P66)

stacksize = 4



空栈

base == top
是栈空标志

top: 非空时, 始终指向栈顶元素的上一个位置。

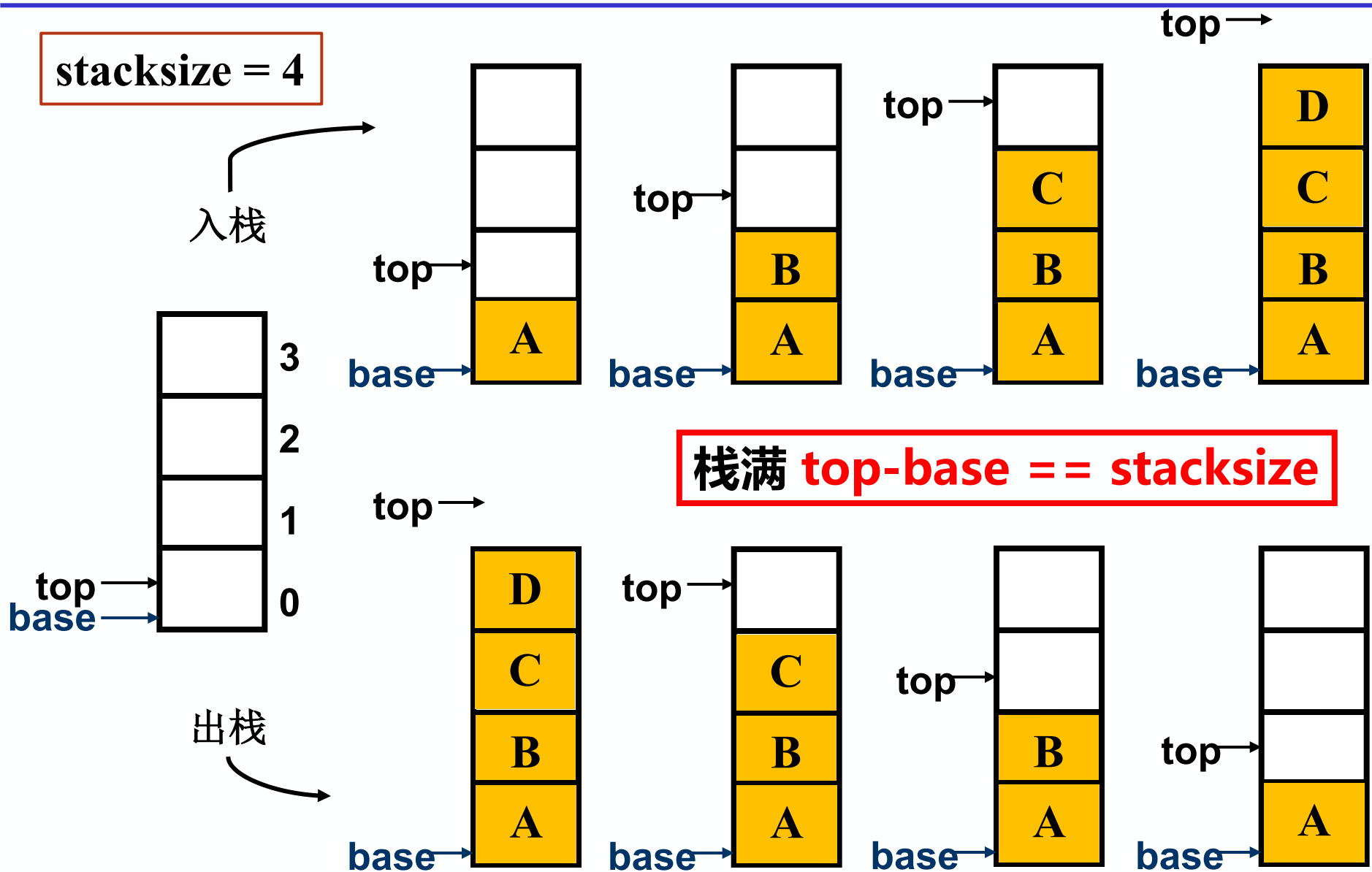
出栈的元素必须在栈顶。

进栈时top增1, 出栈时top减1。

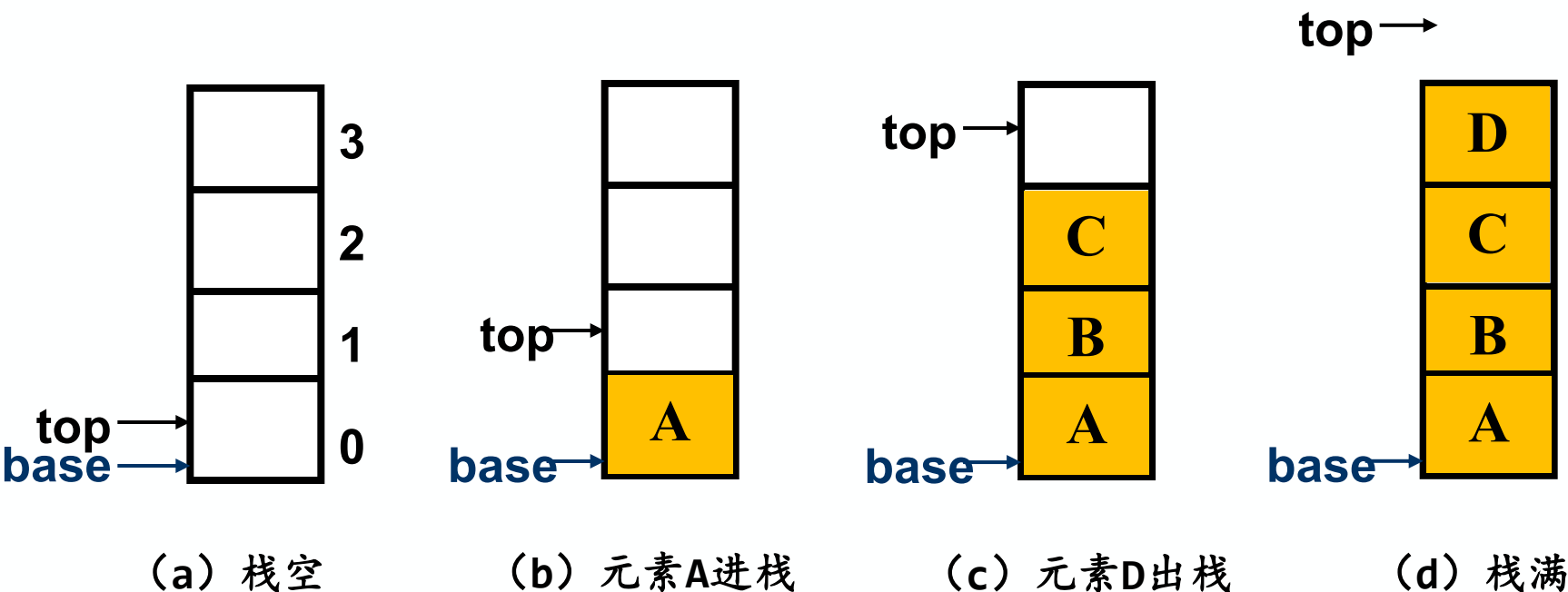
top++

top--

入栈、出栈分解图



顺序栈的各种状态(P66)



栈空: $\text{base} == \text{top}$

入栈: 若栈未满, 将新元素放在 top 处; $\text{top}++$;

出栈: 若栈非空, $\text{top}--$; 从 top 处取出栈顶元素;

栈满: $\text{top} - \text{base} == \text{stacksize}$

【练习1】 设一个栈的输入序列为 a, b, c, d ，则借助一个栈所得到的输出序列不可能是（ ）。

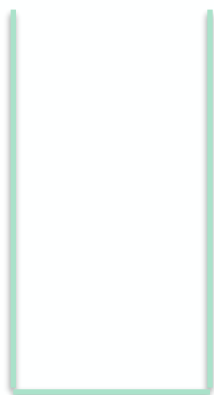
A. c, d, b, a

B. d, c, b, a

C. a, c, d, b

D. d, a, b, c

$d \ c \ b \ a$



栈

下一步不可能出栈 a

答案：D

出栈的元素必须在栈顶

【练习2】若元素a、b、c、d、e、f依次进栈，允许进栈、出栈操作交替进行，但不允许连续3次进行出栈操作，则不可能得到的出栈序列是（ ）。

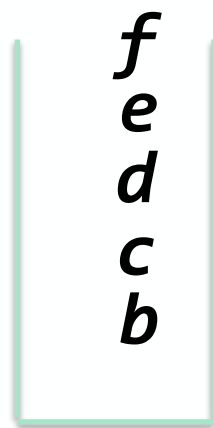
A. dcebfa

B. cbdaef

C. bcaefd

D. afedcb

f e d c b a



连续出栈5次

答案：D

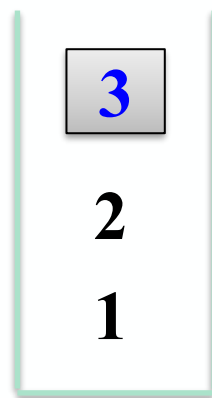
栈

【练习3】 一个栈的入栈序列为1、2、3、...、 $n-1$ 、 n ，其出栈序列是 p_1 、 p_2 、 p_3 、...、 p_n 。若 $p_1=3$ ，则 p_2 可能取值的个数是多少？

- A. $n-3$ B. $n-2$ C. $n-1$ D. 无法确定

1、2、3进栈， 3出栈之后的结果：

4 5 ... n



2 4 5 ... n

$p_2=?$

不可能是1和3
共有 $n-2$ 可能

答案： B

全国考研题

一个栈的入栈序列为 $1, 2, 3, \dots, n$ ，其出栈序列是 $p_1, p_2, p_3, \dots, p_n$ 。若 $p_2=3$ ，则 p_3 可能取值的个数是多少？

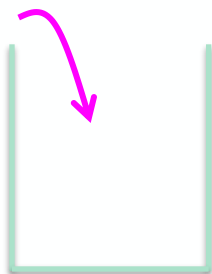
A. $n-3$

B. $n-2$

C. $n-1$

D. 无法确定

$n \quad \dots \quad 3 \quad 2 \quad 1$



不可能是3

解： p_3 可以取1：1进，2进，2出，3进，3出，1出， $\dots$ 。
 p_3 可以取2：1进，1出，2进，3进，3出，2出， $\dots$ 。
 p_3 可以取4：1进，1出，2进，3进，3出，4进，4出， $\dots$ 。
 p_3 可以取5：1进，1出，2进，3进，3出，4进，5进，5出， $\dots$ 。

$\dots$

p_3 可以取除了3外的任何值，共有 $n-1$ 种可能。答案为C。

栈的基本操作(P65)

- **InitStack(&S):** 初始化栈。构造一个空栈S。
- **StackEmpty(S):** 判断栈是否为空：若栈S为空，则返回真；否则返回假。
- **StackLength(S):** 求顺序栈的实际长度。
- **GetTop(S):** 取栈顶元素。若栈S非空，返回当前的栈顶元素，不修改栈顶指针。
- **Push(&S, e):** 进栈。若栈S未满，将元素e插入到栈S中作为栈顶元素。
- **Pop(&S, &e):** 出栈。若栈S非空，从栈S中弹出栈顶元素，并将其值赋给e。

顺序栈的类型定义(P66)

```
#define MAXSIZE 100 //定义栈中元素的最大个数
typedef struct
{
    SElemType *base; //栈底指针
    SElemType *top; // 栈顶指针
    int stacksize; // 栈实际可使用的容量
}SqStack; /*Sequence Stack 顺序栈*/

                S.base

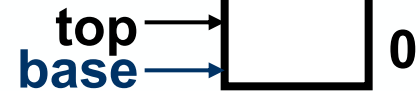
SqStack S; //根据类型建立栈 S    S.top
                S.stacksize
```

顺序栈的初始化(P66)

```
Status InitStack( SqStack &S )
```

```
{
```

```
    S.base = new SElemType[MAXSIZE];
```



//为顺序栈分配一个最大容量为MAXSIZE的数组空间，并用指针base指向这段空间的栈底。

```
    if( !S.base )      exit(OVERFLOW); //存储分配失败
```

```
    S.top = S.base; //栈顶指针等于栈底指针
```

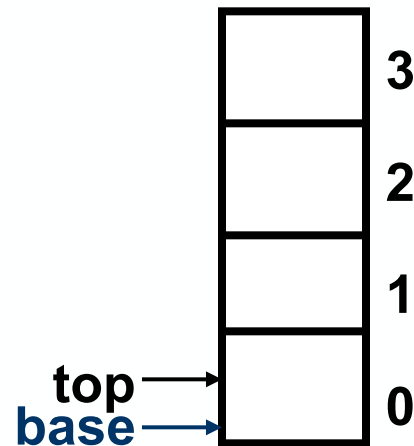
```
    S.stacksize = MAXSIZE;
```

```
    return OK;
```

```
}
```

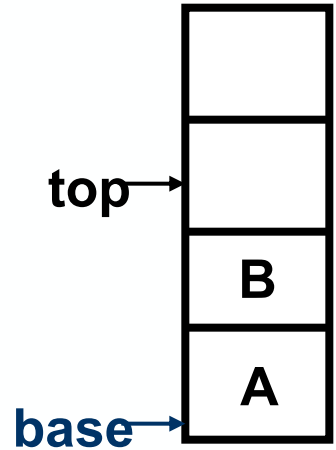
判断顺序栈是否为空

```
Bool StackEmpty( SqStack S ) {  
    //若栈为空, 返回TRUE, 否则返回FALSE  
    if( S.top == S.base )  
        return TRUE;  
    else  
        return FALSE;  
}
```



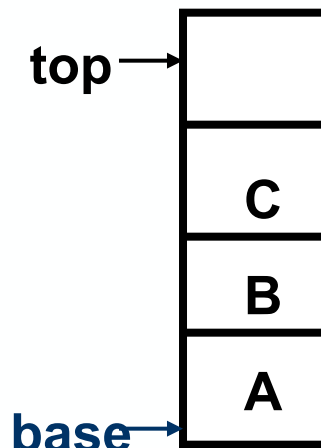
求顺序栈的实际长度

```
int StackLength( SqStack S )  
{  
    return S.top - S.base;  
}
```



顺序栈进栈 (P67)

- (1)判断是否栈满，若满则出错
- (2)元素e压入栈顶
- (3)栈顶指针top加1



```
Status Push( SqStack &S, SElemType e)
```

```
{
```

```
    if( S.top - S.base == S.stacksize )
```

// 栈满

```
        return ERROR;
```

```
    *S.top++=e;
```

```
    return OK;
```

```
}
```

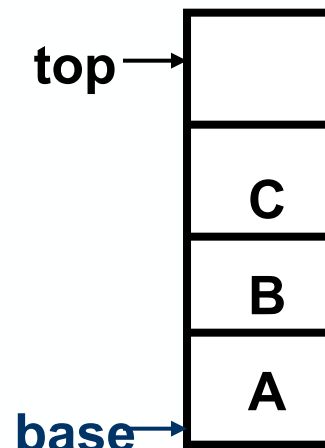
*S.top=e;
S.top++;

针对指针*运算，即对指针所指的空间操作



顺序栈出栈 (P67)

- (1)判断是否栈空, 若空则出错
- (2)栈顶指针减1
- (3)获取栈顶元素, 通过引用参数e返回



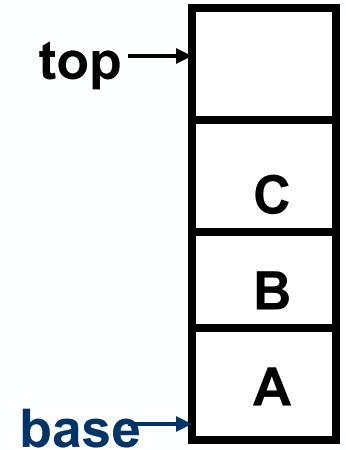
```
Status Pop( SqStack &S, SElemType &e)
```

```
{  
    if( S.top == S.base )    // 栈空  
        return ERROR;  
    e = *--S.top;  
    return OK;  
}
```

--S.top;
e=*S.top;

读取顺序栈的栈顶元素值 (P67)

- (1) 判断是否空栈，若空则返回错误
- (2) 否则通过栈顶指针获取栈顶元素



```
SElemType GetTop( SqStack S)
{
    if( S.top == S.base ) return ERROR; // 栈空，栈下溢出
    else return *( S.top - 1 ); // 返回栈顶元素值，栈顶指针不变
}
```

【练习6】设有一个栈，栈顶元素的下标为1000，每个元素需1个存储单元，在执行Push、Push、Pop、Push、Pop、Push、Pop、Push操作后，栈顶元素的下标为（ ）

A. 1001

B. 1004

C. 1002

D. 1003

答案：C

【练习7】 经过以下栈的操作后，变量x的值为（ ）。
InitStack(S);Push(S,a);Push(S,b);Pop(S,x);GetTop(S,x);

A. a

B. b

C. NULL

D. FALSE

答案：A

【练习8】若一个栈的输入序列是1、2、3、...、 n ，输出序列的第一个元素是 n ，则第 i 个输出元素是（ ）。

A. 不确定

B. $n-i$

C. $n-i-1$

D. $n-i+1$

第 n 个输出元素是（？）

答案：D

解释：第 n 个先出栈，说明前 $n-1$ 个元素已经按顺序入栈先进后出，因此输出序列一定是输入序列的逆序。

第1个元素： n

第 n 个元素： 1

第 i 个元素： $n+1-i$

【练习9】若一个栈的输入序列是1、2、3、...、n，输出序列的第一个元素是i，则第j个输出元素是（ ）。

A. $i-j-1$

B. $i-j$

C. $j-i-1$

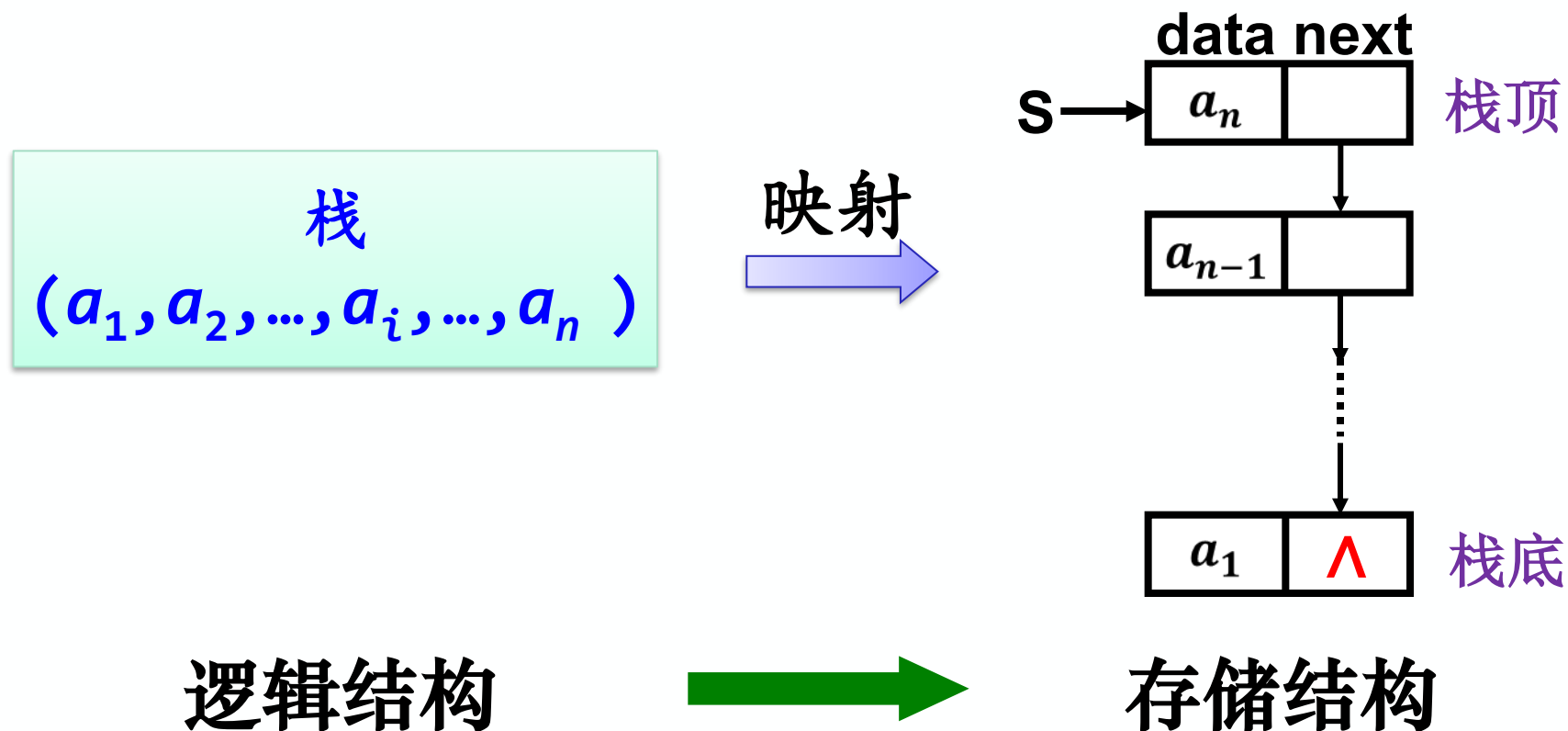
D. 不确定

答案：D

解释：i不定，所以j不定。

3.3.3 链栈的表示和实现(P68)

- ✓ 采用链表存储的栈称为**链栈**，是**运算受限**的单链表，只能在链表顶部进行插入删除操作，故**没有**必要附加**头结点**。栈顶指针就是链栈的头指针

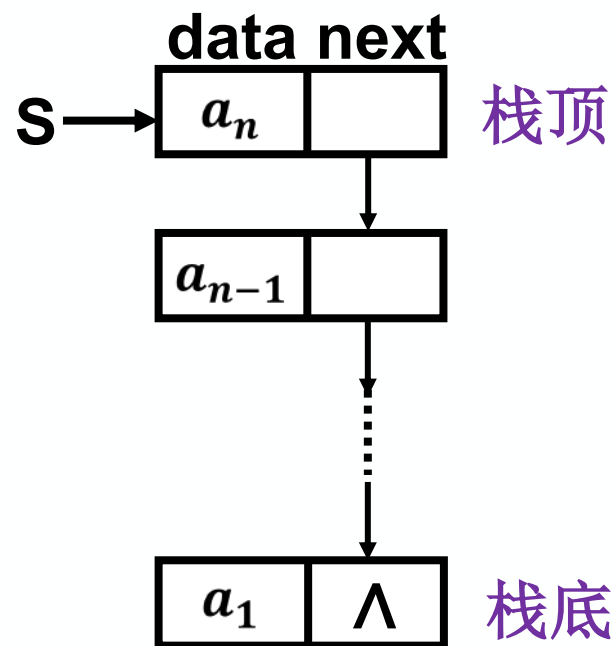


链栈的表示 (P68)

```
typedef struct StackNode
{
    SElemType data; //数据域
    struct StackNode *next; //指针域
} StackNode, *LinkStack;

LinkStack S; //指针类型定义栈顶指针
StackNode *S;

S→data          S→next
```



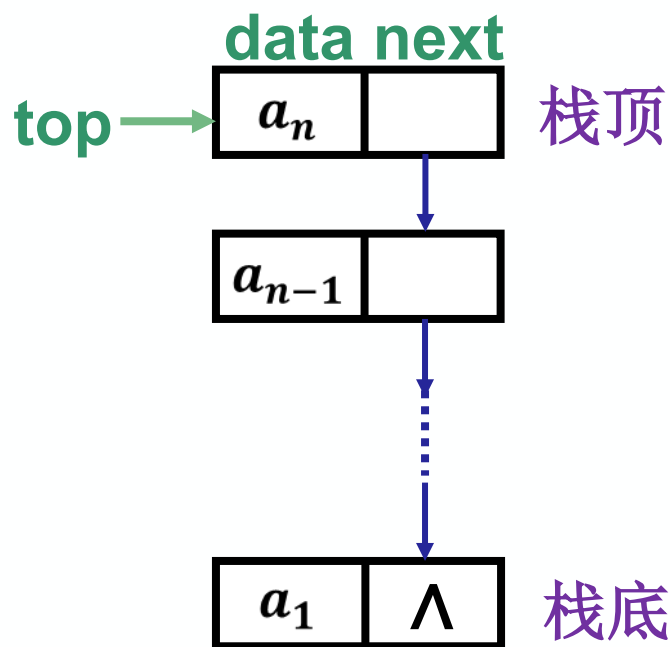
由于栈插入、删除元素在栈顶，因此链栈中指针的方向是相反的， a_n 中存储的是 a_{n-1} 所在结点的地址，元素逆序存放。

链表的头指针就是栈顶指针，且不需要头结点，基本不存在栈满的情况。

【练习10】链式栈结点为：(data,next), top 指向栈顶。若想摘除栈顶结点，并将删除结点的数据域保存到 x 中，则应执行操作 ()。

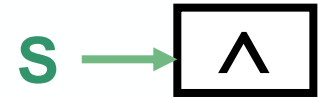
- A. $x=top; top=top \rightarrow next;$
- B. $top=top \rightarrow next; x=top \rightarrow data;$
- C. $x=top \rightarrow data; top=top \rightarrow next;$
- D. $x=top \rightarrow next; top=top \rightarrow data;$

答案：C



链栈初始化 (P68)

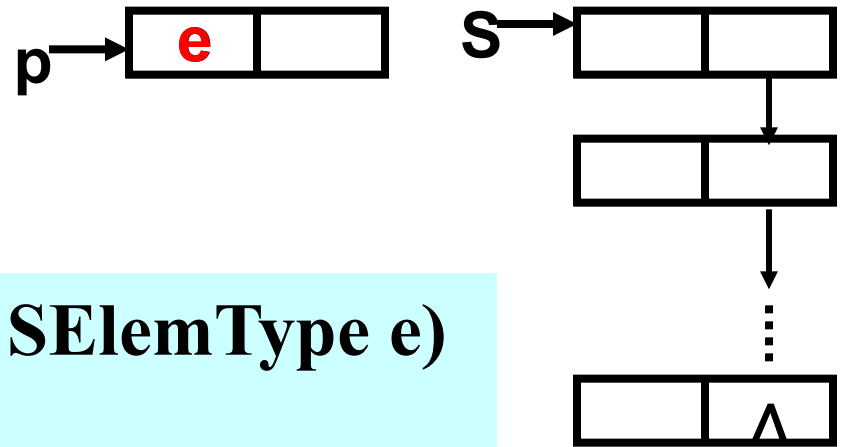
```
Status InitStack( LinkStack &S ){  
    //构造一个空栈, 栈顶指针置为空  
    S = NULL;  
    return OK;  
}
```



链栈判栈空

```
Status StackEmpty( LinkStack S ){  
    if( S == NULL ) return TRUE;  
    else return FALSE;  
}
```

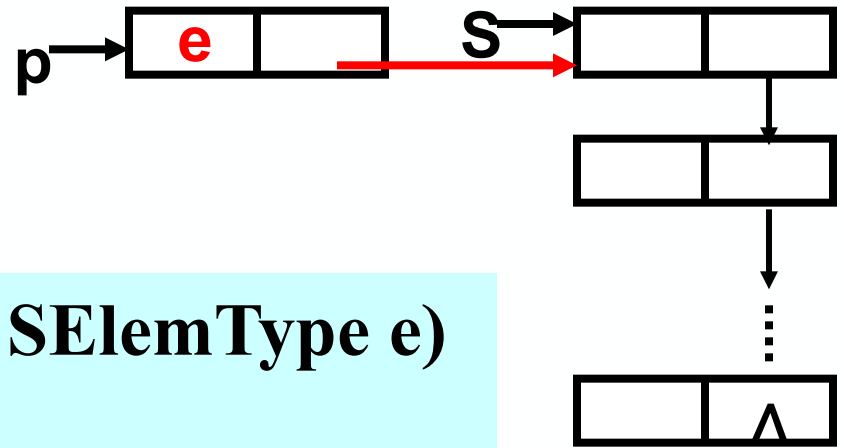
链栈进栈 (P69)



```
Status Push(LinkStack &S , SElemType e)
{
    p=new StackNode; //生成新结点p
    if (!p)    exit(OVERFLOW);

    return OK;
}
```

链栈进栈 (P69)



```
Status Push(LinkStack &S , SElemType e)
```

```
{
```

```
    p=new StackNode;    //生成新结点p
```

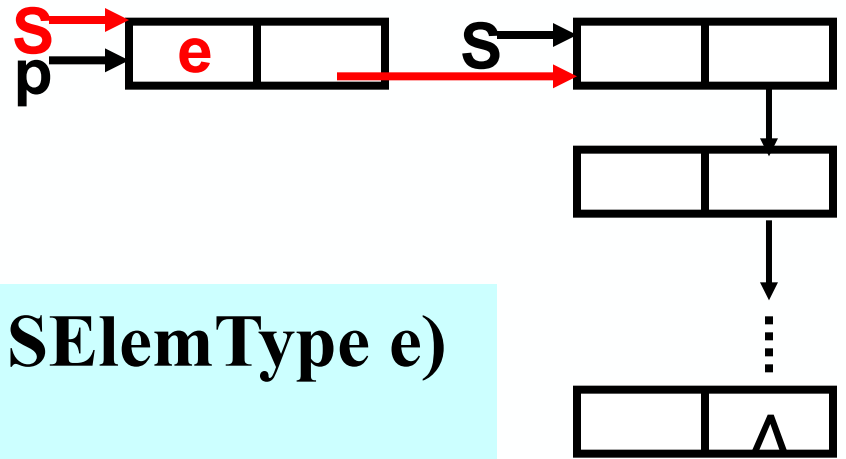
```
    if (!p)      exit(OVERFLOW);
```

```
    p→data=e;    //将新结点数据域置为e
```

```
    return OK;
```

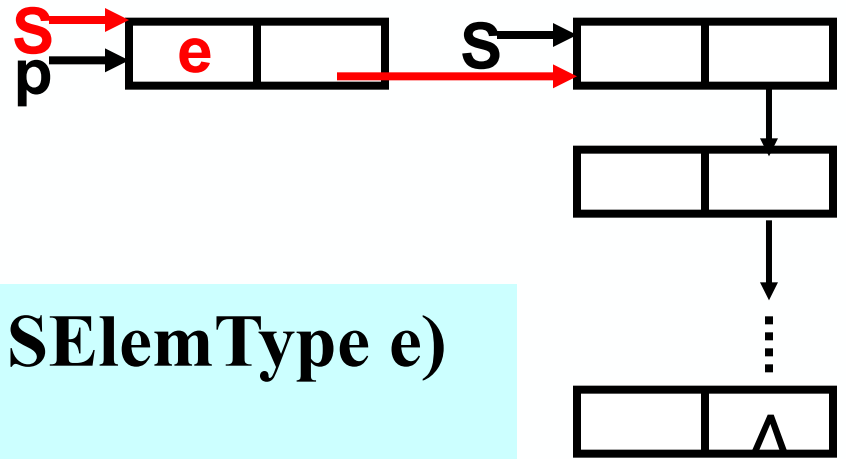
```
}
```

链栈进栈 (P69)



```
Status Push(LinkStack &S , SElemType e)
{
    p=new StackNode;
    if (!p)    exit(OVERFLOW);
    p→data=e; p→next=S; //插入新结点
    return OK;
}
```

链栈进栈 (P69)



```
Status Push(LinkStack &S , SElemType e)
```

```
{
```

```
    p=new StackNode; //生成新结点p
```

```
    if (!p)      exit(OVERFLOW);
```

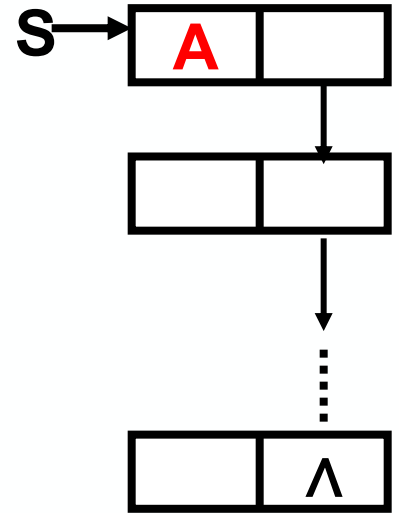
```
    p→data=e;   p→next=S;   S=p;
```

```
    //修改栈顶指针: 栈顶指针S指向新结点, 将S赋值给p
```

```
    return OK;
```

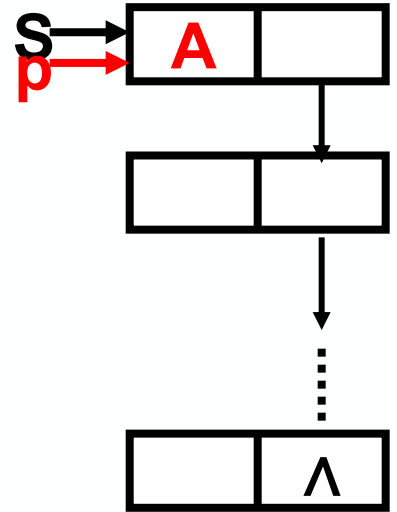
```
}
```


链栈出栈 (P69)



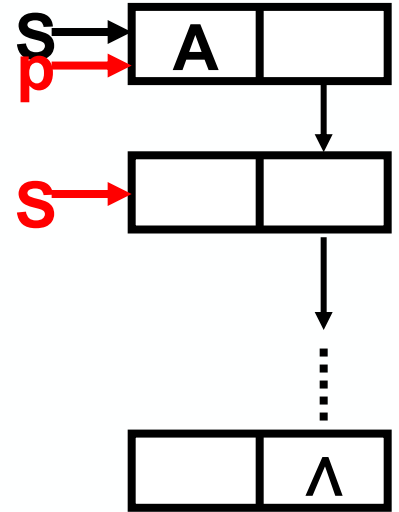
```
Status Pop (LinkStack &S, SElemType &e) {  
    if ( $S == \text{NULL}$ ) return ERROR; //若链栈空, 无法出栈  
  
    return OK;  
}
```

链栈出栈 (P69)



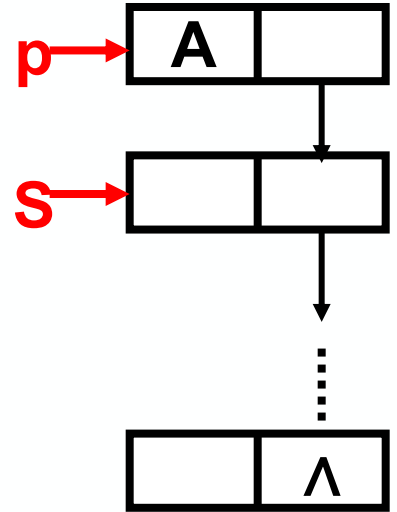
```
Status Pop (LinkStack &S, SElemType &e) {  
    if (S==NULL) return ERROR;  
    e = S→data; //将原栈顶指针的数据域保留在变量e中  
    return OK;  
}
```

链栈出栈 (P69)



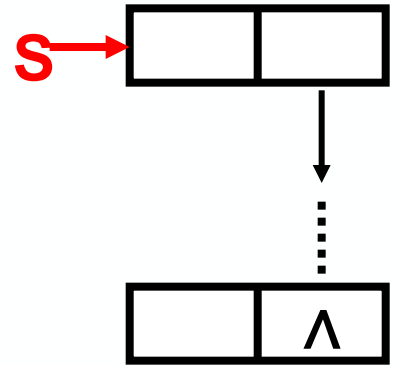
```
Status Pop (LinkStack &S, SElemType &e) {  
    if (S==NULL) return ERROR;  
    e = S→data; p = S; //修改指针  
    return OK;  
}
```

链栈出栈 (P69)



```
Status Pop (LinkStack &S, SElemType &e) {  
    if (S==NULL) return ERROR;  
    e = S→data; p = S; S = S→next;  
    return OK;  
}
```

链栈出栈 (P69)



```
Status Pop (LinkStack &S, SElemType &e) {  
    if (S==NULL) return ERROR;  
    e = S→data; p = S; S = S→next; delete p;  
    return OK;  
}
```

取链栈栈顶元素 (P69)

- (1) 判断是否空栈, 若空则返回错误
- (2) 否则通过栈顶指针获取栈顶元素

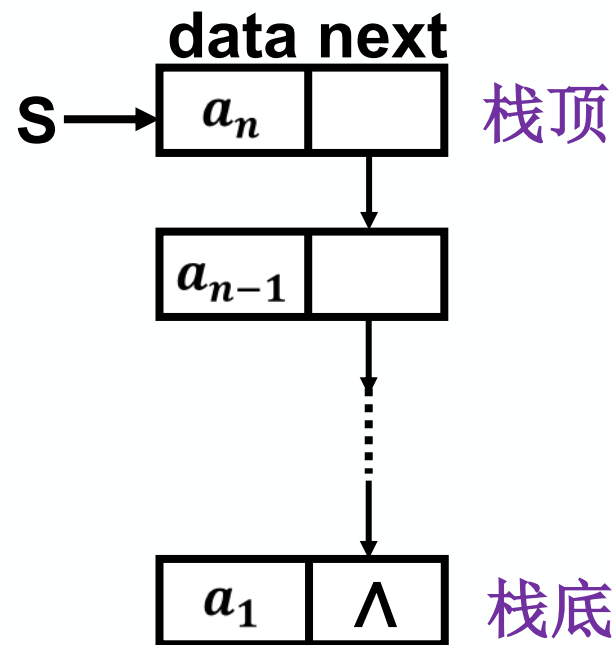
```
SElemType GetTop( LinkStack S)
```

```
{
```

```
    if( S != NULL )           // 栈非空
```

```
        return S→data; // 返回栈顶元素值, 栈顶指针不变
```

```
}
```

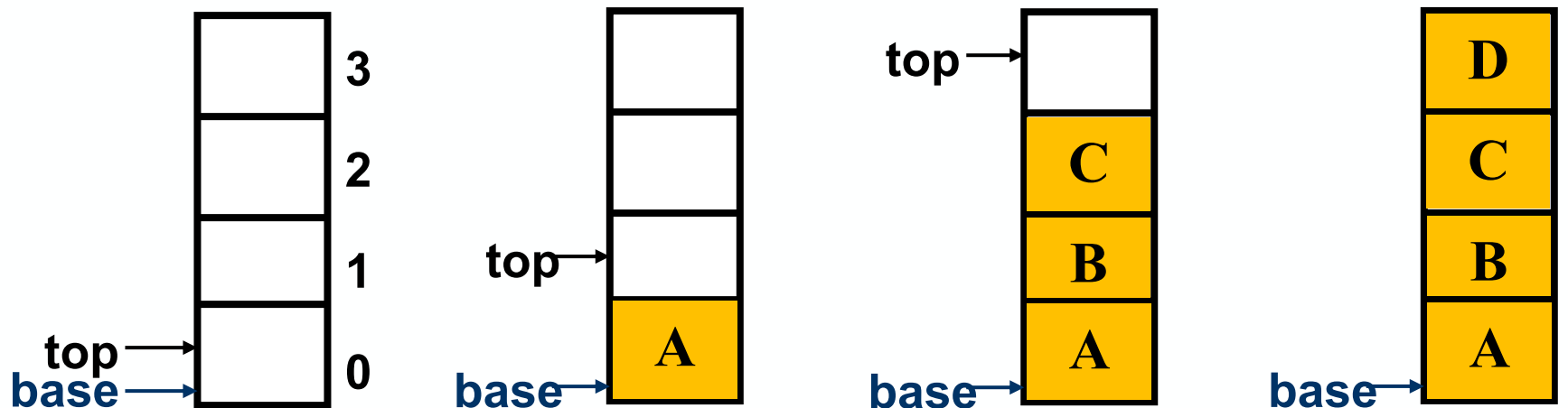


栈的总结

(1) 特点: **先进后出、后进先出**

只能在一端 (**栈顶**) 进行插入和删除运算的**线性结构**

(2) 顺序栈四种状态:



(a) 栈空

(b) 元素A进栈

(c) 元素D出栈

(d) 栈满

栈空: $\text{base} == \text{top}$

入栈: 若栈未满, 将新元素放在top处; $\text{top}++$;

出栈: 若栈非空, $\text{top}--$; 从top处取出栈顶元素;

栈满: $\text{top} - \text{base} == \text{stacksize}$

栈的总结

(3) 顺序栈操作

初始化: $S.base = \text{new SElemType}[\text{MAXSIZE}];$
 $S.top = S.base; \quad S.stacksize = \text{MAXSIZE};$

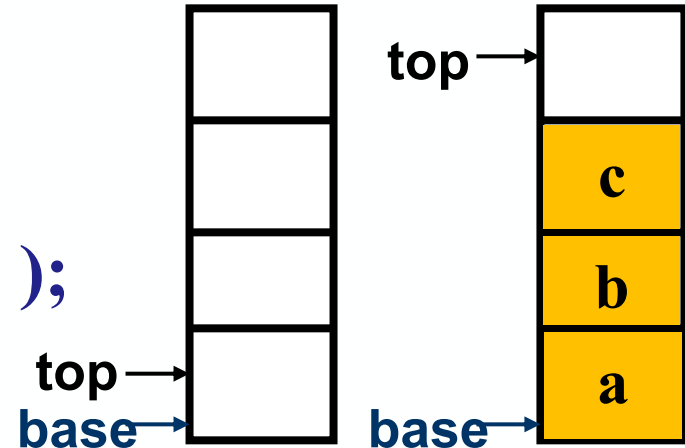
栈空: $S.top == S.base$

实际长度: $S.top - S.base$

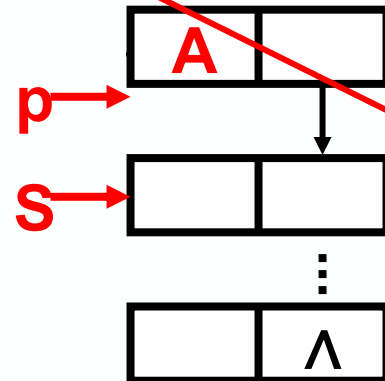
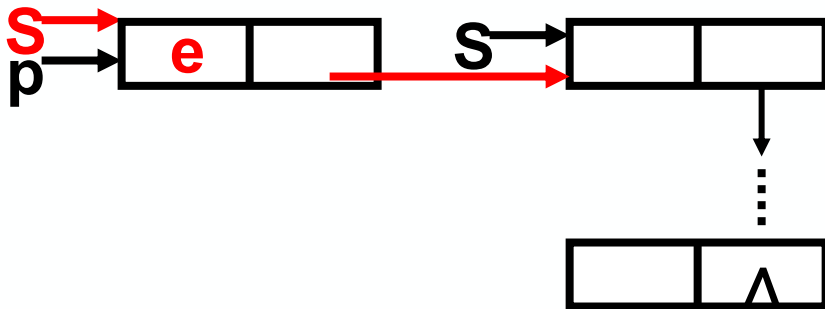
取栈顶元素: $e = *(S.top - 1);$

进栈: $*S.top++ = e;$

出栈: $e = *--S.top;$



(4) 链栈的操作: 进栈, 出栈, 取栈顶元素



栈的应用——表达式

运算符位于操作数中间	● 中缀表达式: $1 + 2 * 3$	} 运算数的相对次序相同
运算符在操作数的后面	● 后缀表达式: $1\ 2\ 3\ * +$	
运算符在操作数的前面	● 前缀表达式: $+ 1\ * 2\ 3$	

中缀 规则: 从左到右计算, 先括号内, 后括号外, 先乘除, 后加减

后缀: 也称逆波兰表达式

人习惯使用中缀表达式, 但计算机使用后缀表达式更方便, 通过**栈**实现。

后缀表达式: 遇到操作数就把操作数压入栈, 遇到运算符就弹出两个操作数, 作为运算符两侧的操作数, 第一个弹出的数在后, 第二个弹出的数在前, 计算后将得到的结果压入栈中。

后缀表达式:
 $123*+$



遇到*号, 弹出3和2, 计算 $2*3=6$, 将6压入栈

遇到+号, 弹出6和1, 计算 $1+6=7$, 将7压入栈

最终栈中只有一个数值, 就是后缀表达式的结果: 7

前缀、后缀、中缀表达式的相互转换

后缀转中缀:

从左往右比较, 遇到连续两个操作数, 后接一个运算符, 则结合

abc+*d-

a(b+c)*d-

a*(b+c)d-

a*(b+c)-d

ab+c*d+eg+h*-

(a+b)c*d+eg+h*-

(a+b)*cd+eg+h*-

((a+b)*c+d)eg+h*-

((a+b)*c+d)(e+g)h*-

((a+b)*c+d)(e+g)*h-

((a+b)*c+d)-(e+g)*h

前缀、后缀、中缀表达式的相互转换

前缀转中缀：

从右往左比较，遇到连续两个操作数，前接一个运算符，则结合

$-*a+bcd$

$-*a(b+c)d$

$-a*(b+c)d$

$a*(b+c)-d$

前缀、后缀、中缀表达式的相互转换

中缀表达式 $a*(b+c)-d$

转前缀:

①每一对运算的外侧加括号

$((a*(b+c))-d)$

②将每个运算符移到对应括号的左边

$-(*(a+(bc)))d$

③去括号

$-*a+bcd$

转后缀:

①每一对运算的外侧加括号

$((a*(b+c))-d)$

②将每个运算符移到对应括号的右边

$((a(bc)+)*d)-$

③去括号

$abc+*d-$

3.4 栈与递归 (P70)



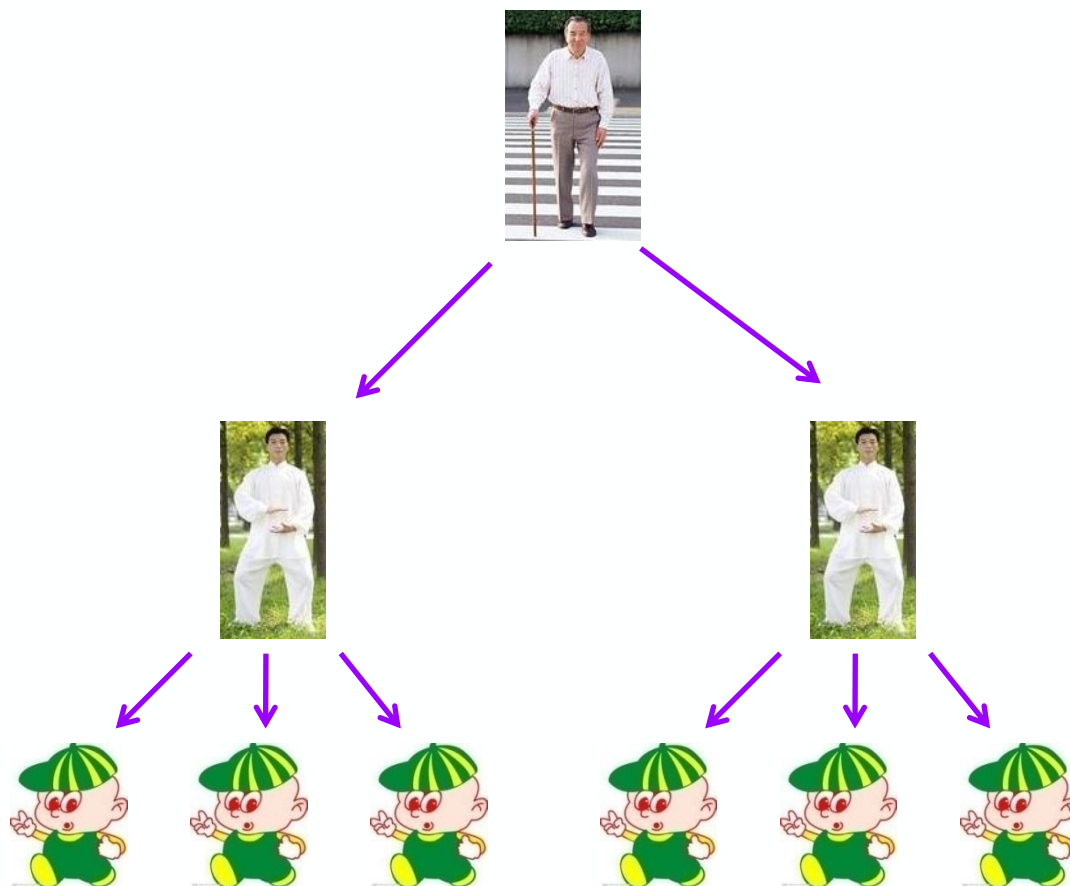
递归的定义

- 若在一个函数、过程或者数据结构定义的内部 直接（或间接）出现定义本身的应用，则称其是递归的或递归定义的。

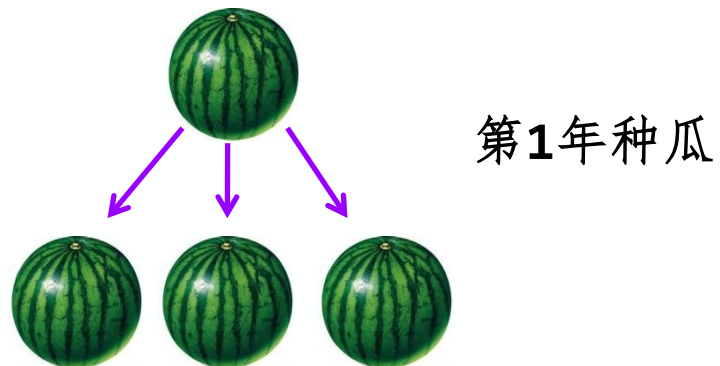
1、对象递归：若一个对象，部分地**包含它自己**，或用**它自己给自己定义**。

举例：链表指针域的定义、树型结构
P30 P137

生活中的实例1：家谱



实例2：种瓜得瓜



第 n 年种瓜



2、过程递归：若一个方法/函数直接或间接地调用自己。

■ 例：递归求正整数n的阶乘

$$Fact(n) = \begin{cases} 1 & \text{若 } n = 0 \text{ 或 } 1 \\ n \cdot Fact(n-1) & \text{若 } n \geq 2 \end{cases}$$

```
long Fact ( long n ) {  
    if ( n == 0 || n == 1 ) return 1;  
    else return n * Fact (n-1);  
}
```

当n=0或1时，返回1，否则返回 $n * Fact(n-1)$

当n为2,返回 $2 * Fact(1) = 2 * 1 = 2$

当n为3,返回 $3 * Fact(2) = 3 * 2 = 6$

...

-
- **以下三种情况常常用到递归方法**
 - **递归定义的数学函数**
 - **具有递归特性的数据结构**
 - **可递归求解的问题**

1. 递归定义的数学函数 (P70)

- 阶乘函数:

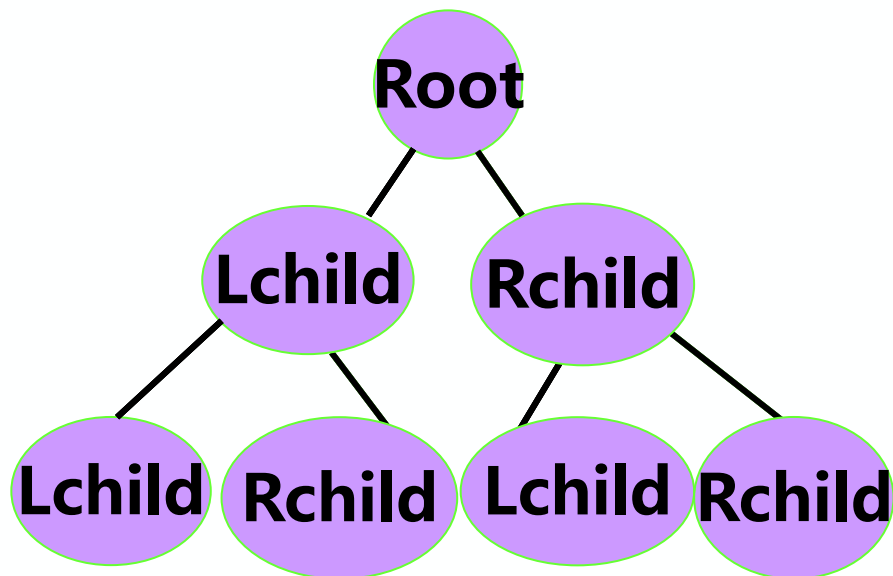
$$Fact(n) = \begin{cases} 1 & \text{若 } n = 0 \text{ 或 } 1 \\ n \cdot Fact(n - 1) & \text{若 } n \geq 2 \end{cases}$$

- 2阶Fibonacci数列:

$$Fib(n) = \begin{cases} 1 & \text{若 } n = 1 \text{ 或 } 2 \\ Fib(n - 1) + Fib(n - 2) & \text{其它} \end{cases}$$

2. 具有递归特性的数据结构:

- 树

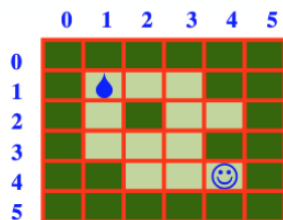


- 广义表

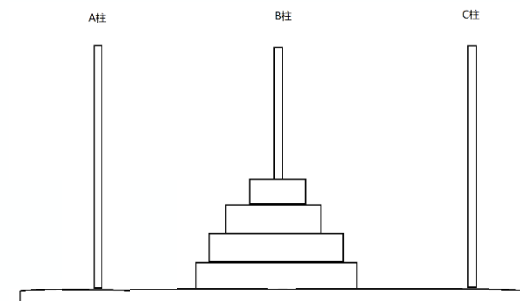
$A = (a, A)$

3. 可递归求解的问题:

- 迷宫问题



Hanoi塔问题



递归问题——分治法求解

分治法： 对于一个较为复杂的问题，能够分解成几个相对简单的且解法相同或类似的子问题来求解。

分治法求解递归问题算法的一般形式：

```
函数返回类型  函数名p (参数列表) {  
    if    (递归结束条件) 可直接求解步骤；  ----基本项  
    else  p (较小的参数)；  -----归纳项  
}
```

```
long Fact ( long n ) {  
    if ( n == 0 || n==1 ) return 1;  //基本项  
    else return n * Fact (n-1);      //归纳项  
}
```

3.5 队列的定义、特点和操作(P77)



若5个人排队
过独木桥



只能按上桥
的次序过桥



这里独木桥就是一个队列！

队列的主要特点是“**先进先出**”，即先入队的元素先出队。队列也称为**先进先出表**。

队列——先进先出

排队上车

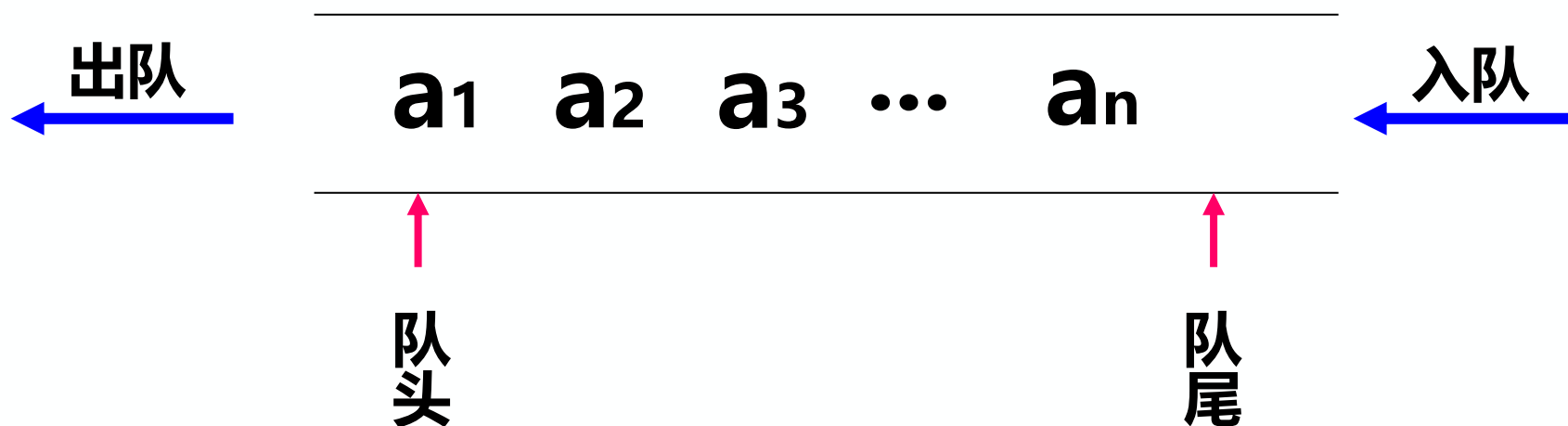


3.5 队列的定义、特点和操作

队列 (Queue)：一种**先进先出(FIFO)**的线性结构。只能在**一端 (队尾)**进行插入，在另一端**(队头)**进行删除。

头 删 尾 插

$$Q = (a_1, a_2, \dots, a_n)$$



栈与队列的异同

对比维度	栈 (Stack)	队列 (Queue)
操作端	只能在 栈顶 操作	只能在 队尾入、队头出
核心规则	先进后出 (FILO)	先进先出 (FIFO)
存储结构	顺序栈或链栈均可	顺序队列或链队列均可
生活场景	叠盘子、弹夹、浏览器后退	买早餐排队、银行叫号、打印机排队
核心操作	入栈 (push) 、出栈 (pop)	入队 (EnQueue) 、出队 (DeQueue)

【练习1】 以下数据结构中，（ ）是非线性数据结构。

- A. 树**
- B. 字符串**
- C. 队列**
- D. 栈**

答案：A，树是一对多的数据结构

【练习2】队列的先进先出特性，是指（ ）。

- 1. 最后插入队列中的元素，总是最后被删除**
- 2. 当同时进行插入、删除操作时，总是插入操作优先**
- 3. 每当有删除操作时，总要先做一次插入操作**
- 4. 每次从队列中删除的总是最早插入的元素**

A. 1

B. 1和4

C. 2和3

D. 1和2和4

答案： B

解释： 队列特点——先进先出、后进后出

【练习3】 设栈S和队列Q的初始时空，元素 e_1 、 e_2 、 e_3 、 e_4 、 e_5 和 e_6 依次通过栈S，一个元素出栈后即进队列。若6个元素出队列的序列是 e_2 、 e_4 、 e_3 、 e_6 、 e_5 、 e_1 ，则栈S的容量至少应该是（ ）。

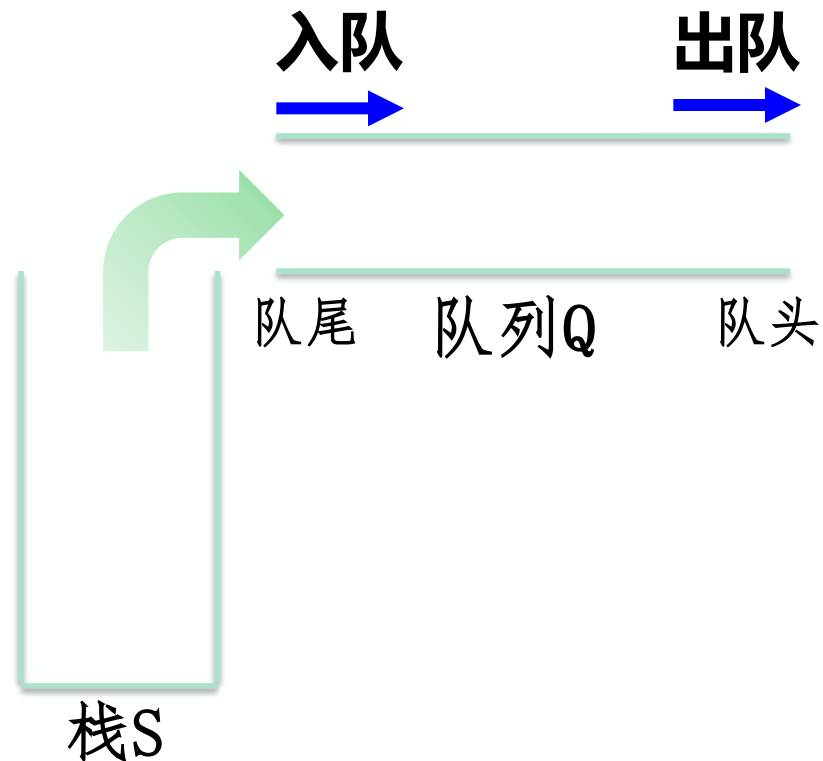
A. 6

B. 4

C. 3

D. 2

e_1 e_2 e_3 e_4 e_5 e_6



答案：C

【练习4】 设栈S和队列Q的初始状态为空，元素abcdefg依次进入栈S。如果每个元素出栈后立即进入队列Q且7个元素的出队顺序是bdcfeag，则栈S的容量至少是（ ）。

A. 1

B. 2

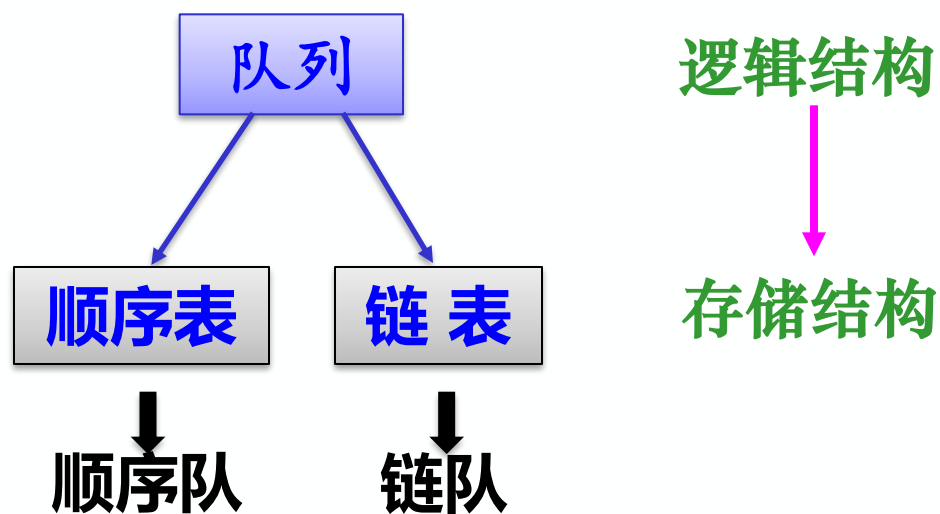
C. 3

D. 4

答案：C

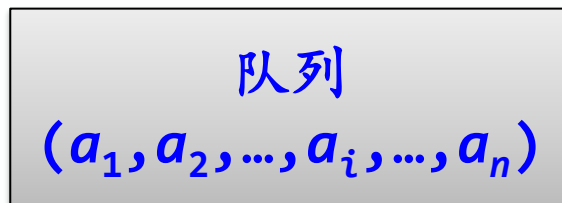
队列的存储结构及其基本运算实现

队列中元素逻辑关系与线性表的相同，队列可以采用与线性表相同的存储结构。



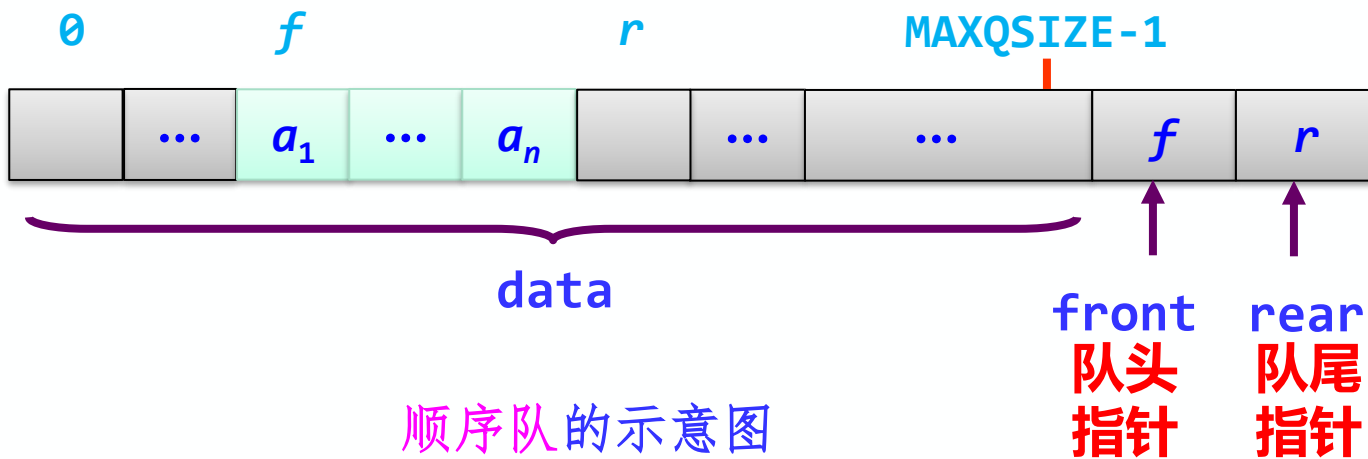
队列的顺序存储结构(P78)

逻辑结构



直接映射

存储结构



顺序队的示意图

因为队列双端都在变化，故需要两个整型变量指示队头元素、队尾元素的位置

队列的顺序表示(P78)

```
#define MAXQSIZE 100 //最大队列长度
```

```
typedef struct{
```

```
    QElemType data[MAXQSIZE]; //初始化分配存储空间
```

```
    int front; //队头指针存放队头元素下标
```

```
    int rear; //队尾指针存放队尾元素后一个位置的下标
```

```
}SqQueue; /*Sequence Queue 顺序队列*/
```

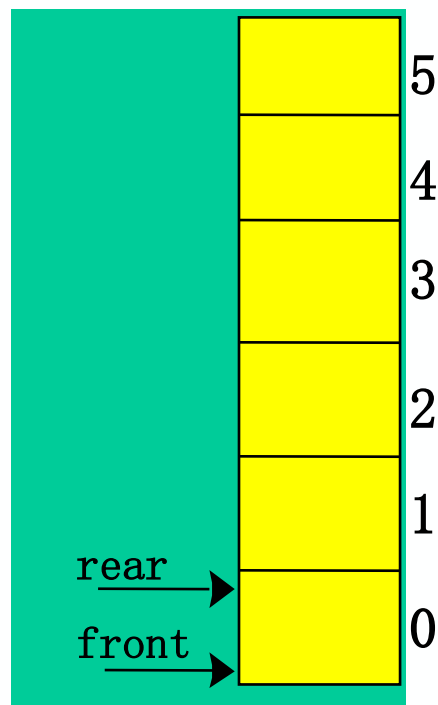
```
SqQueue Q; //根据类型建立队列Q
```

Q.data[下标]

Q.front

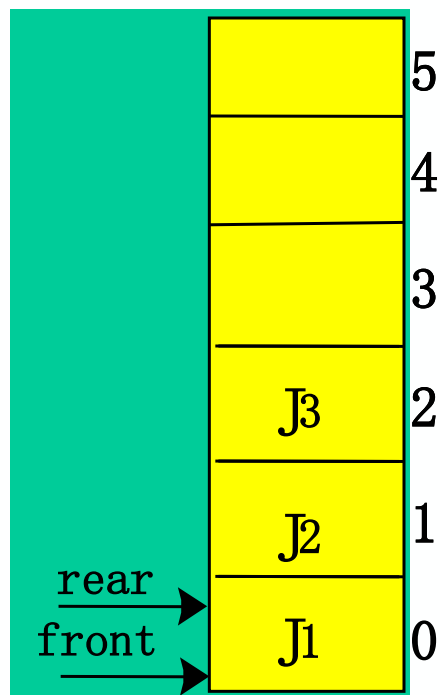
Q.rear

队列的顺序表示——头删尾插

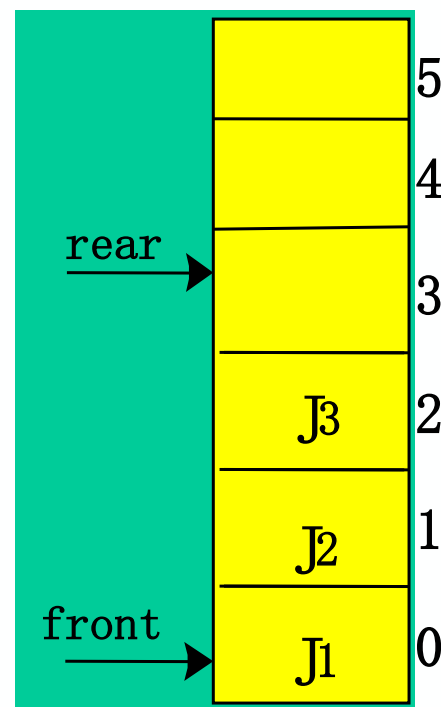


初始状态

`front==rear`



J1、J2、J3入队
元素放到`rear`处;
`rear++`;



J1、J2出队
从`front`处取出元素;
`front++`; J3出队?

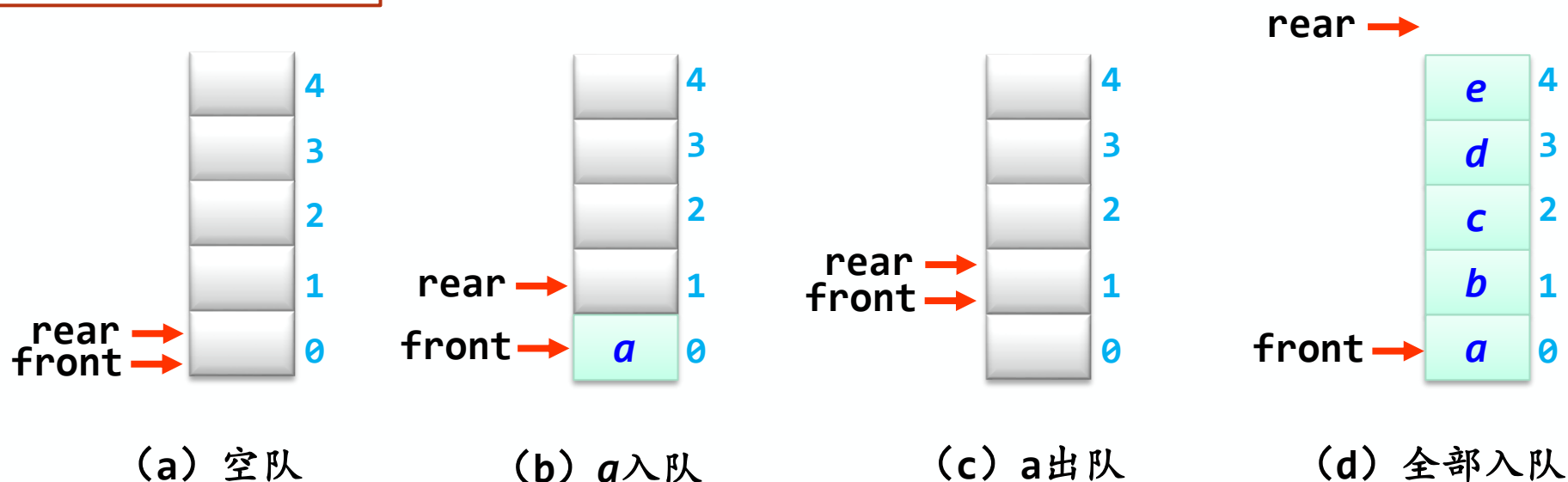
队空

非空队列中, `front`始终指向队头元素
`rear`始终指向队尾元素的后一个位置。

队空:
`front==rear`

队列的各种状态(P78)

MAXQSIZE=5



队空: $\text{front} == \text{rear}$

入队: 若队未满, 将入队元素放在`rear`处; $\text{rear}++$;

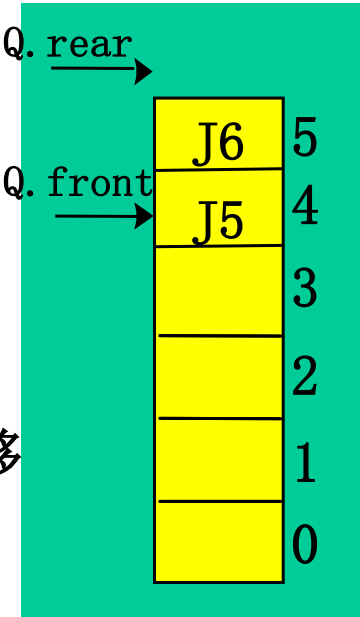
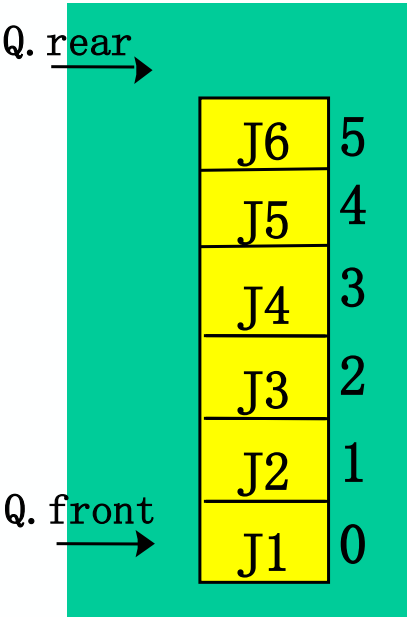
出队: 若队非空, 从`front`处取出队头元素; $\text{front}++$;

队满: $\text{rear} - \text{front} == \text{MAXQSIZE}$

???

存在的问题1：假溢出

MAXQSIZE=6



入队：若队未满，将元素放在rear处；rear++；

还有四个位置，为何不能入队？



**当front==0
再入队J7 真溢出**

**当front≠0
再入队J7 假溢出**

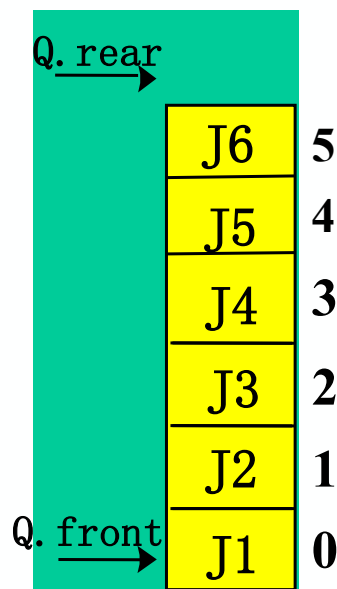
透过现象看本质

第二种情况 新元素无法插入在rear指针所指的位置，但队中还有若干空位置，这种溢出并不是真正的溢出。

解决方法——移动元素

将队中元素依次向队头方向移动。（类似食堂打饭排队）

缺点：浪费时间，每删除一次，队中元素都要移动。



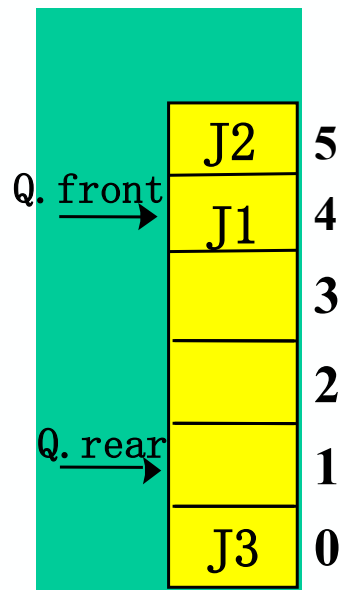
解决方法——循环队列 (P79)

MAXQSIZE=6

(1) 解决假溢出可以将顺序队列想象成一个**环状空间**，即循环使用分配给队列的存储单元。

(2) 当rear指向最后一个合理下标 (若 $\text{rear}+1 == \text{MAXQSIZE}$) 时，下一个rear指向下标0，再入队新元素。

原本 $5+1 \rightarrow 6$ ，现在 $5+1 \rightarrow 0$



(3) 利用**取余运算** (C语言: %)

$$\begin{aligned} \text{Q.rear} &= (\text{Q.rear} + 1) \% 6 \\ &= (5 + 1) \% 6 = 0 \end{aligned}$$

$$\text{当 rear}=1, (1+1)\%6=2$$

$$\text{当 rear}=2, (2+1)\%6=3$$

$$\text{当 rear}=3, (3+1)\%6=4$$

总结：当Q.rear为MAXQSIZE-1时，为了避免再进队出现假溢出，插入元素可使 $\text{Q.rear} = (\text{Q.rear} + 1) \% \text{MAXQSIZE}$ ，构成循环队列。

解决方法——循环队列

MAXQSIZE=6

取余运算 (mod, C语言: %)

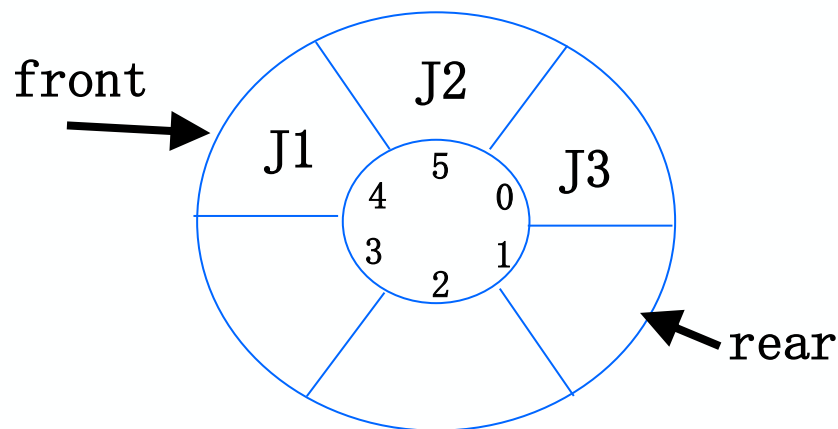
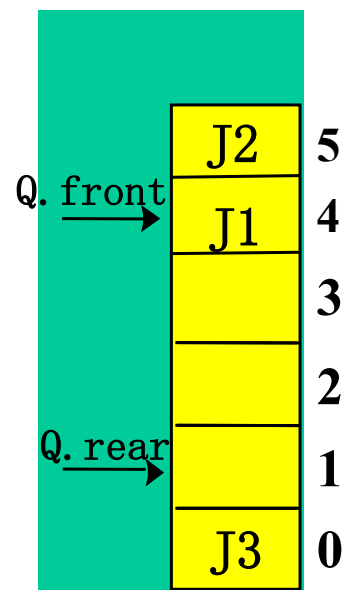
$$\begin{aligned} Q.rear &= (Q.rear + 1) \% MAXQSIZE \\ &= (5 + 1) \% 6 = 0 \end{aligned}$$

插入元素x: $Q.data[Q.rear] = x;$

$Q.rear = (Q.rear + 1) \% MAXQSIZE;$

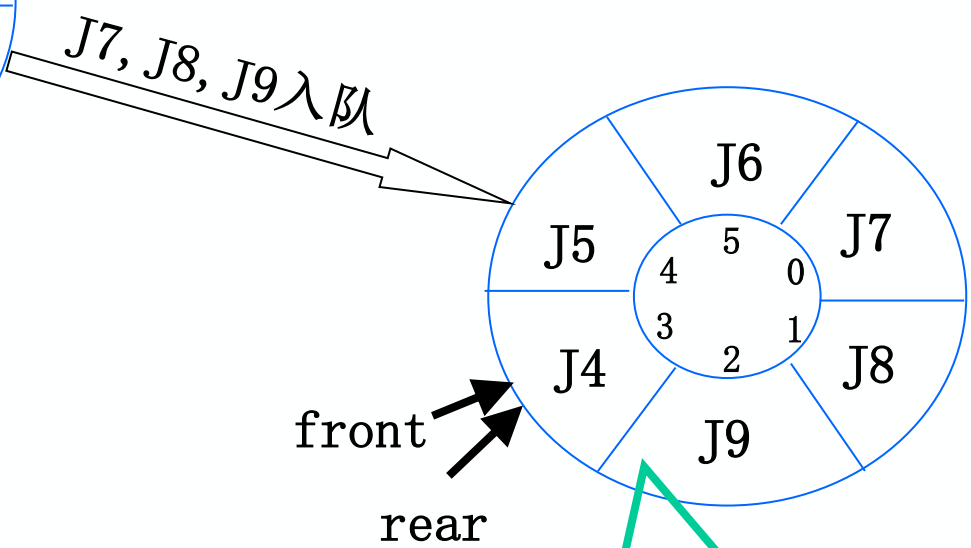
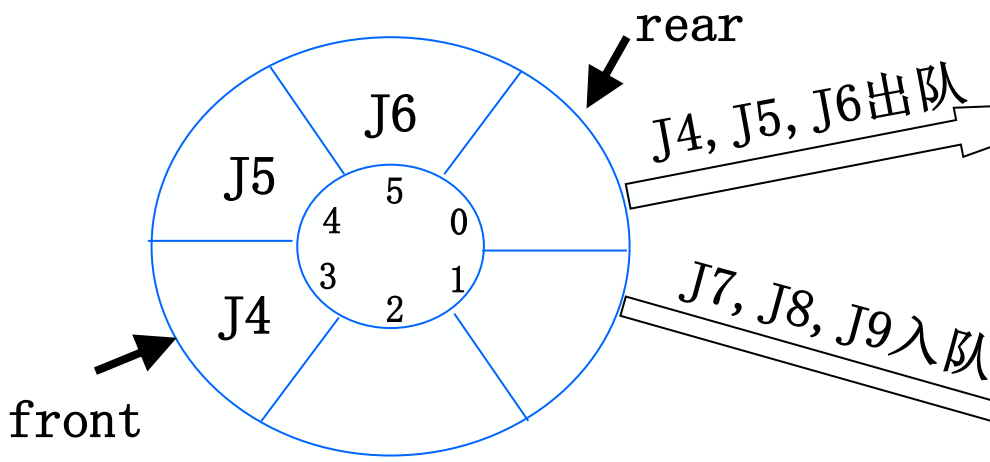
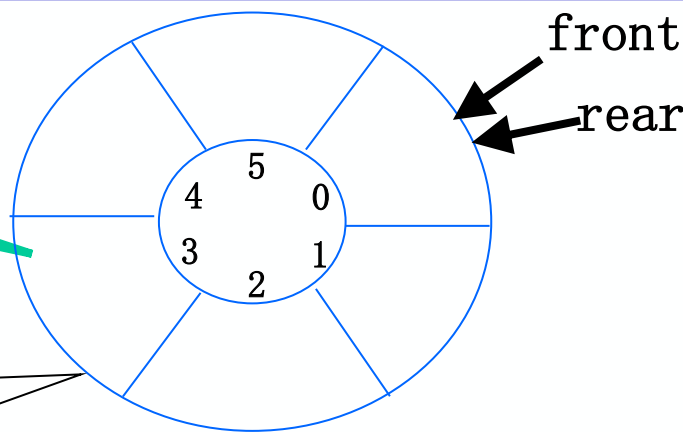
删除元素: $x = Q.data[Q.front];$

$Q.front = (Q.front + 1) \% MAXQSIZE;$



存在的问题2：队空和队满条件相同

队空： $front == rear$



队满： $front == rear$

解决方法——循环队列少用一个空间 P79

解决方案:

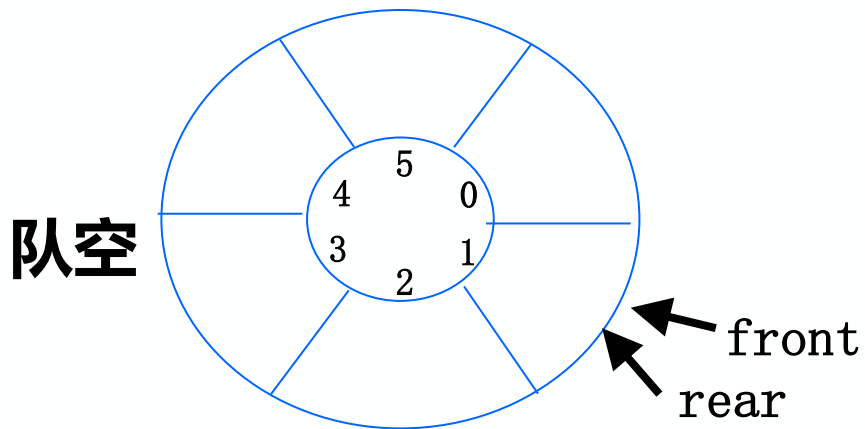
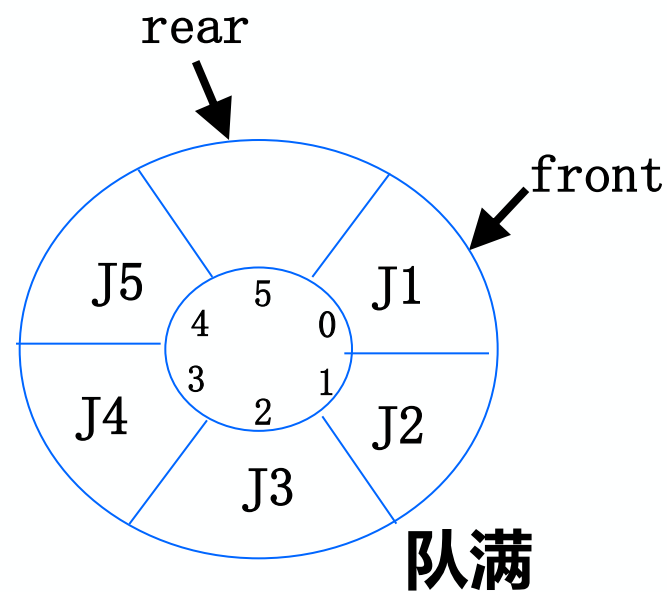
少用一个元素空间即为满

即队列空间大小为M时，有M-1个元素就认为队满。

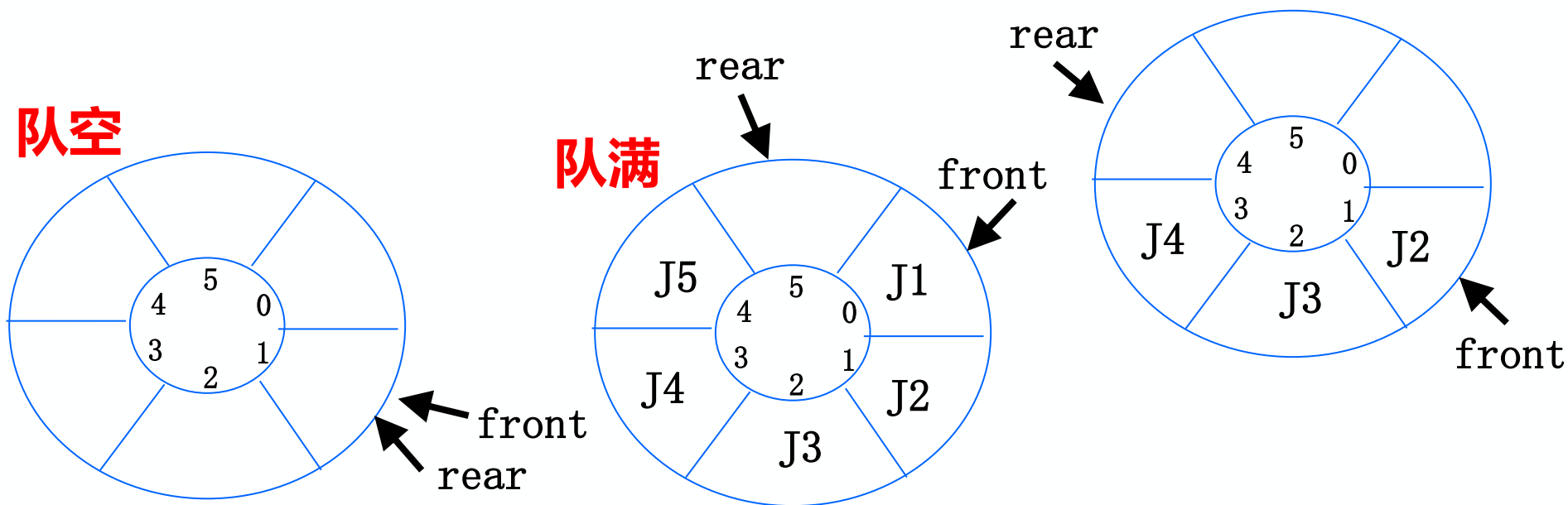
队空: $front == rear$

队满: $(rear + 1) \% M == front$

让 $rear + 1 = front$



循环队列中的各种状态(P79)



队空: $\text{front} == \text{rear}$ **队满:** $(\text{rear} + 1) \% \text{MAXQSIZE} == \text{front}$

入队: 若队未满, 将入队元素放在rear处;
 $\text{rear} = (\text{rear} + 1) \% \text{MAXQSIZE};$

出队: 若队非空, 从front处取出元素;
 $\text{front} = (\text{front} + 1) \% \text{MAXQSIZE};$

循环队列的类型定义(P78)

```
#define MAXQSIZE    100    //最大长度
```

```
typedef struct{
```

```
    QElemType *base; //动态分配存储空间, base存放数组的基地址
```

```
    int front; //头指针, 若队列不空, 指向队头元素
```

```
    int rear; //尾指针, 若队列不空, 指向队尾元素的下一个位置
```

```
}SqQueue;
```

```
SqQueue Q; //根据类型建立队列Q
```

Q.front Q.rear Q.base Q.base[下标]

循环队列初始化(P80)

Status InitQueue (SqQueue &Q)

{

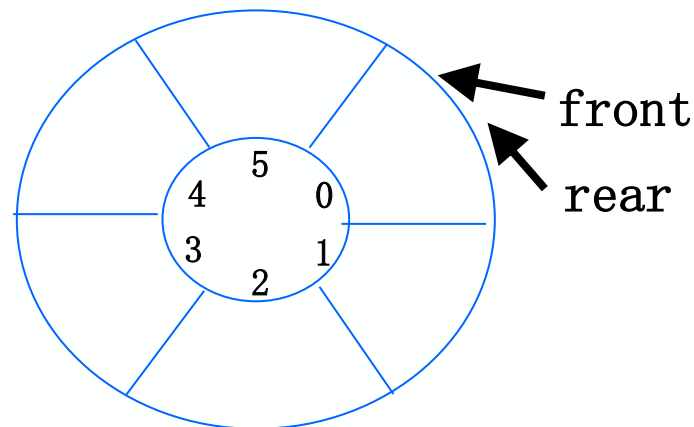
Q.base =new QElemType[MAXQSIZE]; //分配空间

if(!Q.base) exit(OVERFLOW); //存储分配失败

Q.front=Q.rear=0; //头尾指针下标相同，队列为空

return OK;

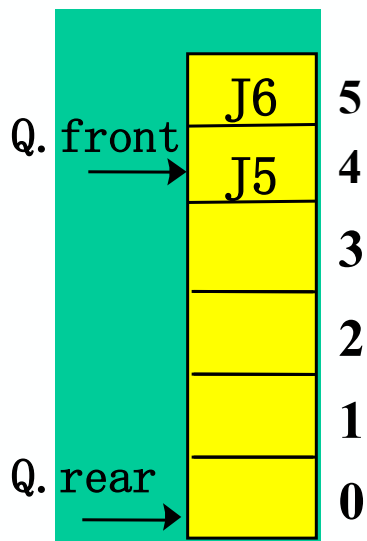
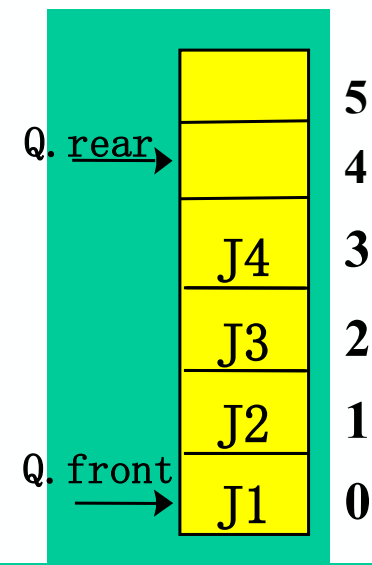
}



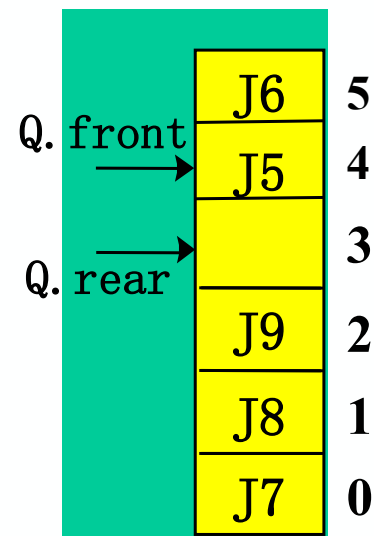
求循环队列的长度——求实际元素个数(P80)

```
int QueueLength (SqQueue Q) {  
    return ( Q.rear-Q.front+MAXQSIZE )%MAXQSIZE;  
}
```

MAXQSIZE=6



对于循环队列，有时
Q.rear比Q.front小
此时差值为负数。
需要加上MAXQSIZE



循环队列入队(P80)

Status EnQueue(SqQueue &Q, QElemType e)

{

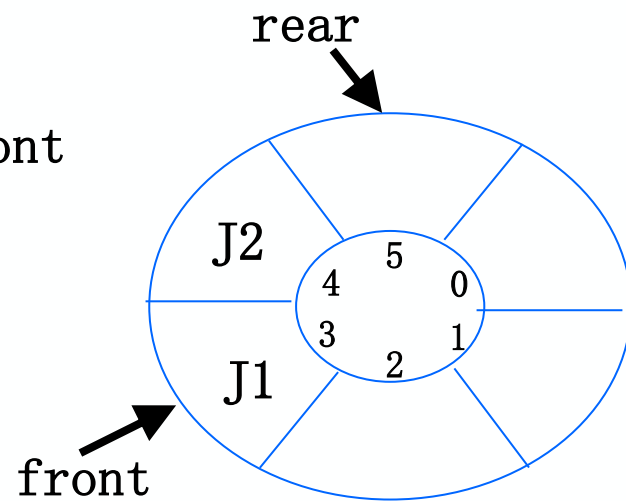
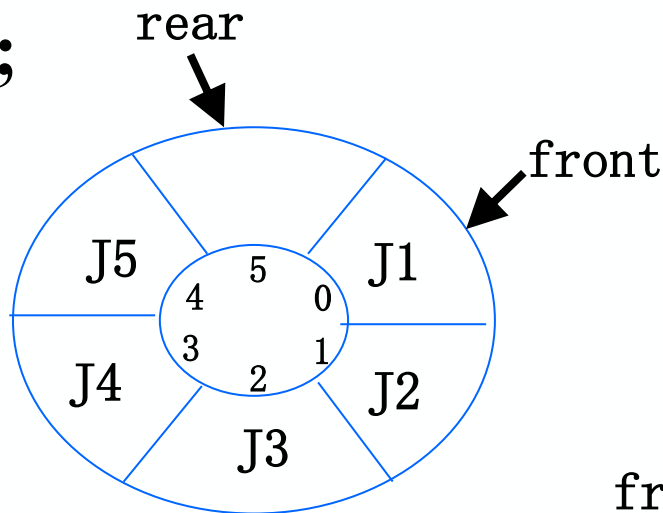
if((Q.rear+1)%MAXQSIZE==Q.front) **return** ERROR; **//队满出错**

Q.base[Q.rear]=e; **//新元素插入到队尾rear所指位置**

Q.rear=(Q.rear+1)%MAXQSIZE; **//队尾指针rear向后移**

return OK;

}



循环队列出队(P81)

Status DeQueue (SqQueue &Q, QElemType &e)

{

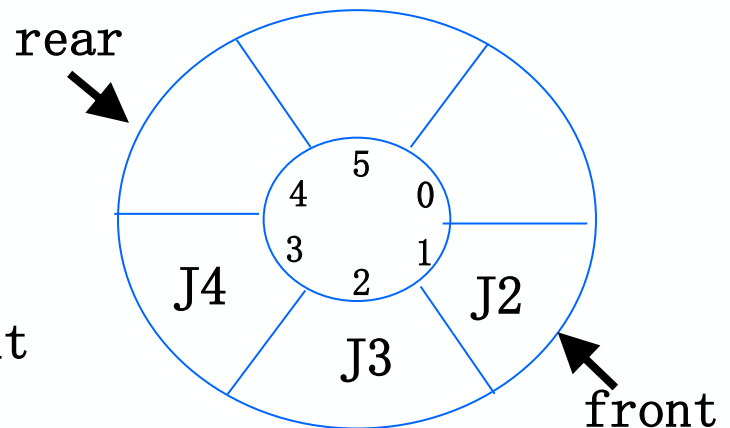
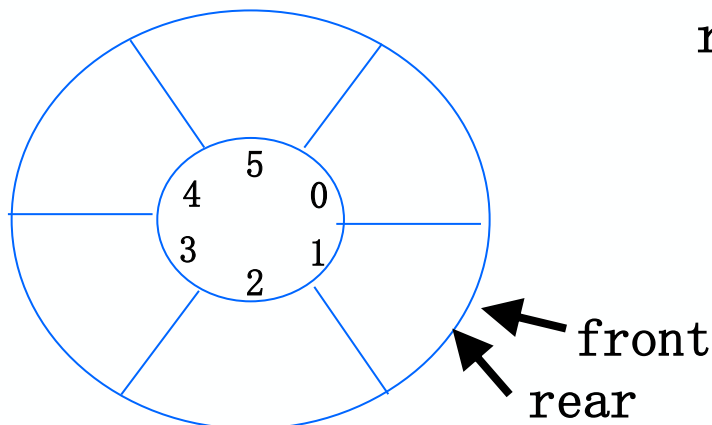
if(Q.front==Q.rear) return ERROR; //队空出错

e=Q.base[Q.front]; //从front处取出元素

Q.front=(Q.front+1)%MAXQSIZE; //front指针指向下一元素

return OK;

}



循环队列存储在数组A[0...n]中，则入队时的操作为 ()

A. $\text{rear} = \text{rear} + 1$

B. $\text{rear} = (\text{rear} + 1) \% (n - 1)$

C. $\text{rear} = (\text{rear} + 1) \% n$

D. $\text{rear} = (\text{rear} + 1) \% (n + 1)$

答案： D

假设一个循环队列Q[MAXSIZE]的队头指针为front，队尾指针为rear，队列的最大容量为MAXSIZE，除此之外，该队列再没有其他数据成员，则判断该队的队满条件为（ ）

- A. $Q.front == Q.rear$
- B. $(Q.front + Q.rear) \geq MAXSIZE$
- C. $Q.front == (Q.rear + 1) \% MAXSIZE$
- D. $Q.rear == (Q.front + 1) \% MAXSIZE$

答案： C

循环队列放在一维数组A[0...M-1]中，end1指向队头元素，end2指向队尾元素的后一个位置。假设队列中最多能容纳M-1个元素，初始时为空。下列判断队空和队满的条件中，正确的是（ ）

- A. 队空： $\text{end1} == \text{end2}$ 队满： $\text{end1} == (\text{end2} + 1) \% (M - 1)$
- B. 队空： $\text{end1} == \text{end2}$ 队满： $\text{end2} == (\text{end1} + 1) \% (M - 1)$
- C. 队空： $\text{end1} == \text{end2}$ 队满： $\text{end1} == (\text{end2} + 1) \% M$
- D. 队空： $\text{end1} == (\text{end2} + 1) \% M$
队满： $\text{end2} == (\text{end1} + 1) \% M$

答案： C 队空： $\text{front} == \text{rear}$
队满： $(\text{rear} + 1) \% \text{MAXQSIZE} == \text{front}$

若用数组A [0...5]来实现循环队列，且当前rear和front的值分别为1和5，当从队列中删除一个元素，再加入两个元素后， rear和front的值分别为（ ）。

A. 3和4

B. 3和0

C. 5和0

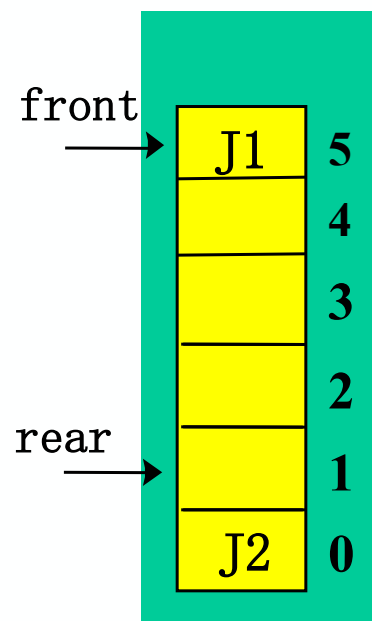
D. 5和1

答案： B

解释： 每删除一个元素，队首指针：

$$\text{front} = (\text{front} + 1) \% 6$$

每插入一个元素，队尾指针： $\text{rear} = (\text{rear} + 1) \% 6$



已知循环队列的存储空间为数组A[21]，假设当前front和rear的值分别为8和3，则该队列的长度为（ ）。

A. 5

B. 6

C. 16

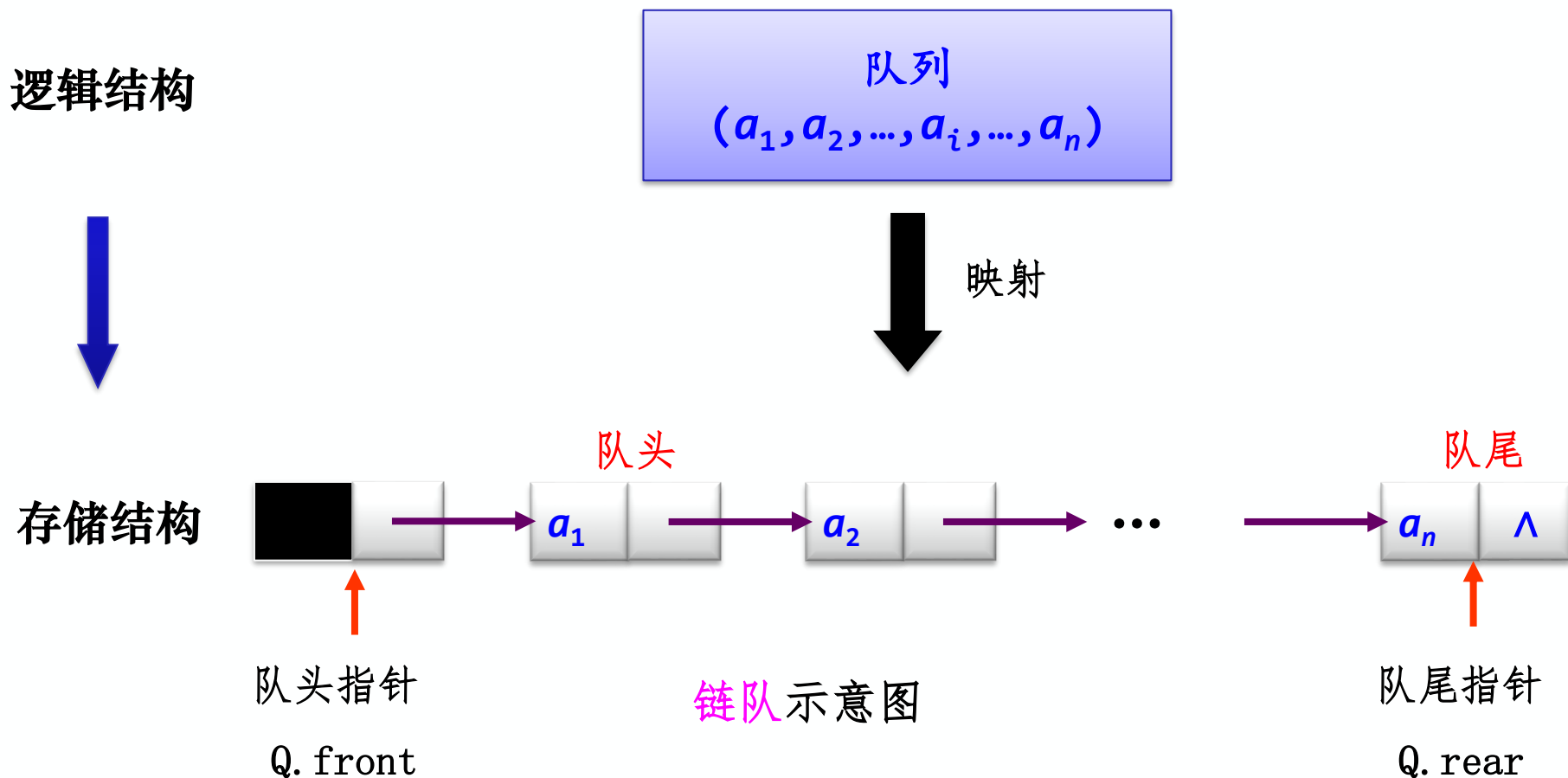
D. 17

答案：C

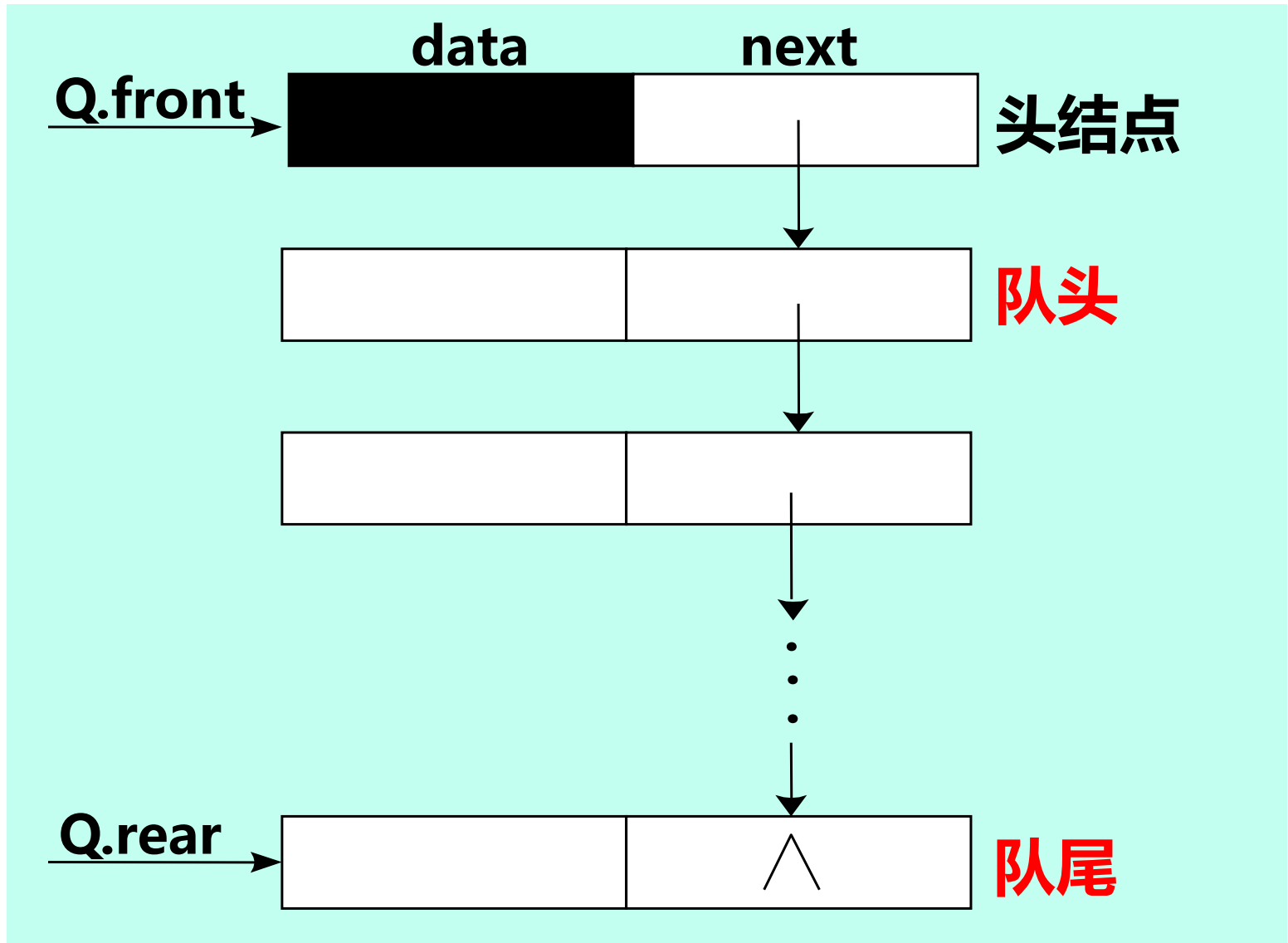
循环队列长度 = $(Q.rear - Q.front + MAXQSIZE) \% MAXQSIZE$
 $= (3 - 8 + 21) \% 21 = 16 \% 21 = 16$

3.5.3 队列的链式存储结构(P81)

若用户无法估计所用队列的长度，或者元素个数变化大，宜采用链式存储的队列。为了操作方便，采用**带头结点的单链表**实现**链队**。



链队列(P81)



链队列的类型定义(P81)

```
#define MAXQSIZE 100 //最大队列长度
```

```
typedef struct QNode { //先定义链队列中的结点类型QNode
```

```
    QElemType    data; //数据
```

```
    struct QNode *next; //指针 (递归定义, 用指针指向的结点类型定义指针)
```

```
} QNode, *QueuePtr; //结点类型, 指向结点的*指针类型
```

```
typedef struct {
```

```
    QueuePtr front;           //队头指针
```

```
    QueuePtr rear;           //队尾指针
```

```
} LinkQueue; //链式队列
```

```
LinkQueue Q;
```

Q.front

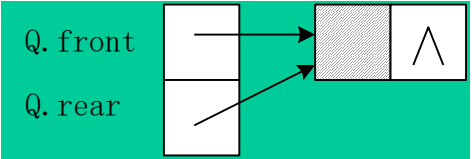
Q.front→next

Q.rear

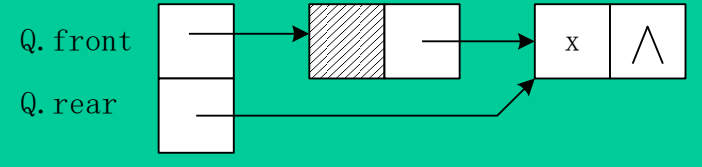
Q.rear→next

链队列 头删尾插(P82)

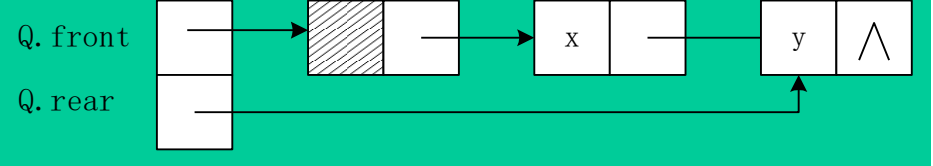
(a) 空队列



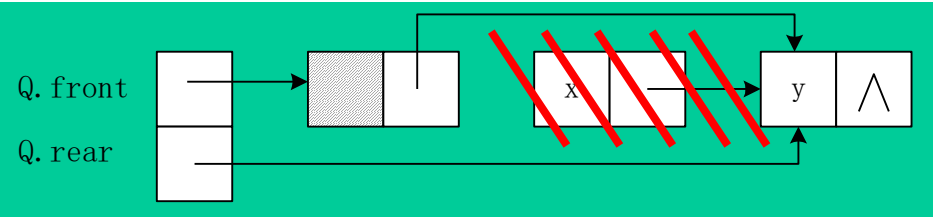
(b) 元素x入队列



(c) 元素y入队列



(d) 哪个元素先出队列?
X



链队列初始化(P82)

Status InitQueue (LinkQueue &Q)

```
{ //生成新结点作为头结点，队头和队尾指向此结点
```

```
Q.front=Q.rear=new QNode;
```

```
if(!Q.front) exit(OVERFLOW); //内存不够，分配失败
```

```
Q.front→next=NULL; //头结点的指针域为空
```

```
return OK;
```

```
}
```



链队列入队 头删尾插(P82)

```
Status EnQueue(LinkQueue &Q, QElemType e) {
```

```
    p=new QNode; //准备新结点
```

```
    if(!p) exit(OVERFLOW); //新结点生成失败
```

```
    p→data=e; p→next=NULL; //准备新结点
```

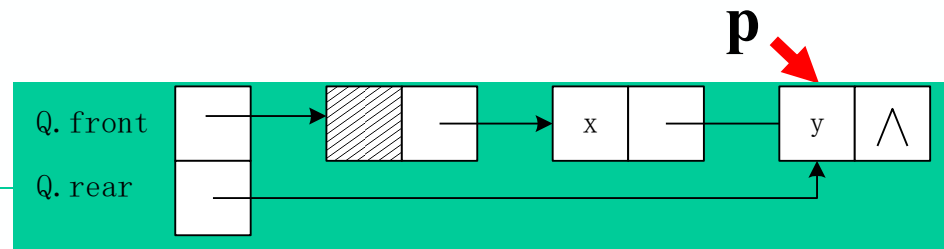
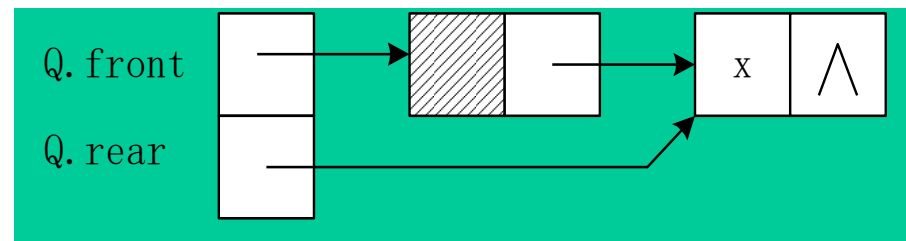
```
    Q.rear→next=p;
```

```
    //修改原本尾指针所指向结点的next域， 指向新结点
```

```
    Q.rear=p; //修改队尾指针
```

```
    return OK;
```

```
}
```



链队列出队 头删尾插(P83)

Status DeQueue (LinkQueue &Q,QElemType &e)

{

if(Q.front==Q.rear) return ERROR;//判队空

p=Q.front→next;

e=p→data;

Q.front→next=p→next;

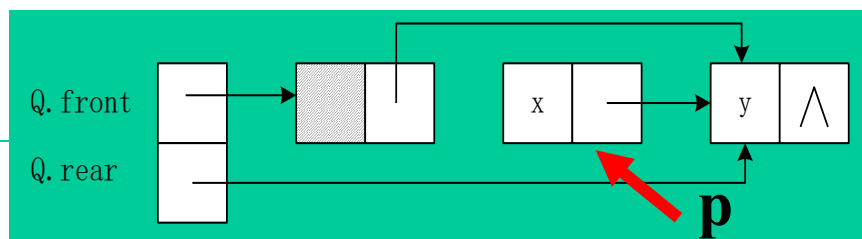
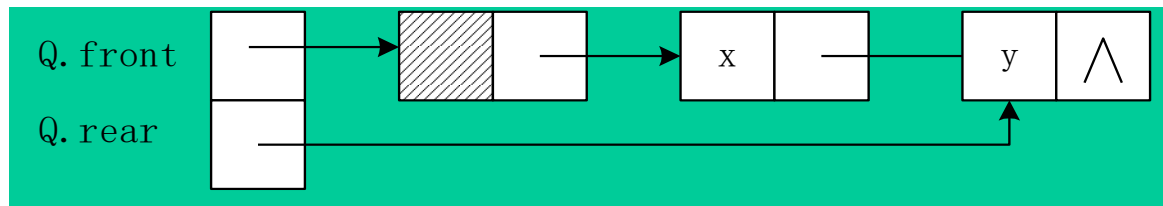
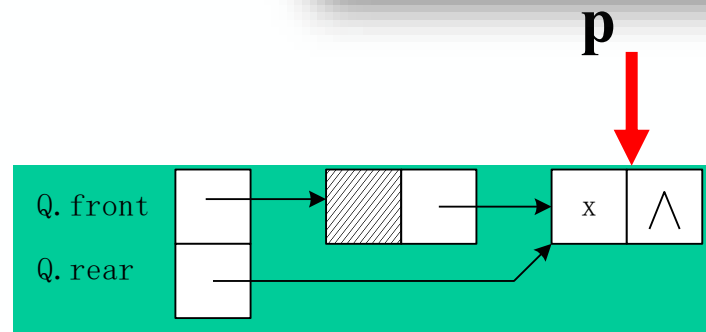
if(Q.rear==p) Q.rear=Q.front;

//如果删的是尾结点, 删除之后队列为空, 不仅要修改头指针, 也要修改尾指针

delete p; //free(p);

return OK;

}



练习一下

1. 有六个元素6, 5, 4, 3, 2, 1 的顺序进栈, 问下列哪一个不是合法的出栈序列? (**C**)
 - A. 5 4 3 6 1 2
 - B. 4 5 3 1 2 6
 - C. 3 4 6 5 2 1
 - D. 2 3 4 1 5 6
2. 循环队列存储在数组A[0..m]中, 则入队时的操作 (**D**) 。
 - A. $rear=rear+1$
 - B. $rear=(rear+1)\bmod(m-1)$
 - C. $rear=(rear+1)\bmod m$
 - D. $rear=(rear+1)\bmod(m+1)$
3. 若用一个大小为6的数组来实现循环队列, 且当前rear和front的值分别为0和3, 当从队列中删除一个元素, 再加入两个元素后, rear和front的值分别为多少? **B**
 - A. 1和 5
 - B. 2和4
 - C. 4和2
 - D. 5和1
4. 栈和队列的共同点是 (**C**) 。
 - A. 都是先进先出
 - B. 都是先进后出
 - C. 只允许在端点处插入和删除元素
 - D. 没有共同点